

The Calculus of Disposition

Processes, Types, and Witnessed Computation

Flyxion
Independent Researcher

September 2026

Working draft: Preface and eleven chapters drafted,
remaining chapters in plan form

Contents

Preface	iii
I Orientation	1
1 Computation with a Past	2
1.1 Four commitments of a calculus	2
1.2 State and history	2
1.3 The lifecycle	3
1.4 Records and the step function	4
1.5 A first example	6
2 Disposition Worlds	7
2.1 Worlds and configurations	7
2.2 A small language	8
2.3 Countermodels	9
2.4 History twins and collapse lenses	10
2.5 Causal shuffles	11
2.6 The witness mixer	11
2.7 Perturbations	12
2.8 Disposition control	12
2.9 The laboratory	14
3 The Kernel and the Disposition Layer	16
3.1 The kernel, restated	16
3.2 Kernel histories and admissibility	17
3.3 Continuation spaces and guards	17
3.4 The translation	18
3.5 Forward simulation and backward adequacy	19
3.6 Opaque and transparent readings	20
3.7 What the translation preserves	20
3.8 Decisions fixed by this chapter	21
4 Encodings and Their Criteria	22

II	Terms and Names	23
5	Syntax and Binding	24
5.1	Two syntactic categories	24
5.2	Free names and substitution	25
5.3	Structural congruence	26
6	Concrete Syntax and Unique Parsing	27
6.1	Lexical classes	27
6.2	The grammar	28
6.3	Unique derivations	28
6.4	Adequacy of the printer	29
6.5	The executable check	29
7	The Lambda Layer	31
8	Combinators, Erasure, and Linear Resource	32
III	Processes and Labelled Transitions	33
9	CCS: Synchronization and Binding	34
10	CSP and the Two Refusals	35
11	The Pi-Calculus: Restriction and Fresh Commitments	36
12	Labelled Disposition Semantics	37
12.1	Actions and records	37
12.2	Primitive rules	37
12.3	Contextual rules	39
12.4	Worked derivations	39
12.5	First preservation results	41
12.6	Authority	43
13	Choice, Actors, and the Arbiter	45
IV	Linear and Session Types	46
14	The Type Algebra	47
14.1	Types, rights, and facts	47
14.2	Operator types	48
14.3	Context algebra	49
14.4	Typing the value layer	49
14.5	Typing processes	51
14.6	Configurations and the rights–state correspondence	52
14.7	The proof target	52

15 Preservation, Progress, and Protocol Fidelity	54
15.1 Structural lemmas	54
15.2 Preservation	55
15.3 Progress	56
15.4 Protocol fidelity	57
15.5 Negative derivations	58
16 Authority as a Modality	59
16.1 Exclusive authority as a linear capability	59
16.2 Delegation	59
16.3 Results	60
16.4 Case study: evidence is not authority	61
V The Disposition Calculus	64
17 The Ledger Path Lemma	65
17.1 The lemma	65
17.2 Causal order	66
18 Refusal, Binding, and Evidence	67
18.1 Refusal preservation	67
18.2 Binding integrity	67
18.3 Witness ancestry	68
19 Collapse Soundness and Terminal Exclusivity	69
20 Compensation	71
VI Replay, Progress, and Concurrency	73
21 Replay	74
21.1 Replay as a fold	74
21.2 Consistency as an invariant	75
21.3 Completeness and anchors	75
21.4 Duplicate delivery	76
21.5 Which reorderings replay ignores	77
22 Petri Nets, Traces, and Event Structures	79
23 Logical Clocks, CRDTs, and the Single Sequencer	80
24 Eventual Disposition	81
24.1 Hypotheses on executions	81
24.2 The theorem	82
24.3 What fails without each hypothesis	82

25 Bisimulation and the Spectrum as Collapse Rules	85
26 Checking the Layer Translation	86
27 Typed Disposition Safety	87
28 A Map of Positions	89
A Proof Spine	90
B Complete Syntax, Typing, and Transition Rules	92
C Encodability Criteria, Stated Formally	93
D A Worked Encoding: A CCS Fragment into the Calculus	94
E Open Problems	95

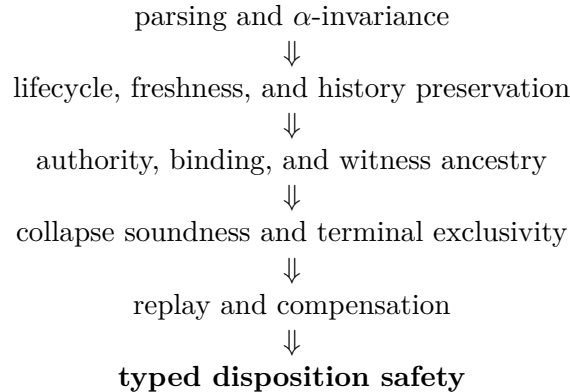
Preface

Every formal calculus of computation decides what a term is, what may happen to it, when two results count as the same, and what an observer is allowed to distinguish. These decisions are often introduced separately. They are not independent. A calculus that treats computation as rewriting will naturally ask which rewritten terms have the same normal form. A calculus that treats computation as communication will instead ask which interactions an environment can observe. A calculus that retains a history must ask a further question: which differences may be ignored without asserting that recorded events never occurred?

The calculus developed in this book begins from the last question. Its basic subject is not a value alone, nor a process considered apart from its past, but a commitment moving through a witnessed history. A commitment may be bound, transformed, verified, refused, or collapsed into an observation. Each such action changes what continuations remain admissible. None is permitted to revise the past that made it possible.

This orientation yields a system with three formal layers and one durable semantic object. A lambda layer computes values, predicates, transformations, reasons, and witnesses. A process layer organizes fresh commitments, interaction, choice, concurrency, and disposition. A type algebra separates reusable knowledge from linear rights to advance a commitment. An append-only ledger records the events from which the current disposition state can be reconstructed. The layers are kept distinct on purpose. Lambda terms compute values; processes consume and produce commitments, capabilities, evidence, and observations. A transformation has a functional type such as $f : A \multimap B$, while a process applies that transformation within the history of one particular commitment.

The principal theorem is therefore not merely that well-typed terms do not go wrong. It is that a commitment reaches a terminal disposition only by an authorized, causally complete, and durably witnessed history. We call this property *typed disposition safety*. The proof is cumulative:



The final proof is short because the book is arranged so that nearly all of its content has already been proved locally. In the present calculus most of the untyped spine turns out to rest on a single invariant, the *ledger path lemma* of Chapter 17: the records of each commitment, read in order, trace a legal path through its lifecycle ending at its current state. Binding integrity, witness ancestry, collapse soundness, terminal exclusivity, and refusal preservation are then corollaries rather than separate theorems. The type algebra of Part IV strengthens the spine in its final form: commitment uniqueness from linearity, lifecycle monotonicity from typestate preservation, and the impossibility of a premature collapse from the input type of **collapse**.

Two layers, one vocabulary

A reader who knows the canonical Spheredpop kernel will notice that the disposition calculus uses its operator names with more specific meanings. In the kernel, **Pop** commits to one option among those available; **Refuse** records that an option is inadmissible without removing it from the history; **Bind** couples two elements without identifying them; and **Collapse_c** observes a history under a rule *c*. In the disposition calculus, **pop** introduces a fresh commitment; **refuse** is one of two terminal dispositions, available from every live state; **bind** associates a commitment with a value or capability; **collapse** produces an observation from a verified commitment under a declared rule *k*; and **transform** and **verify** appear as lifecycle stages.

The book’s claim is that the disposition calculus is a discipline *over* the kernel rather than a separate calculus, and it is a claim to be proved, not assumed. Chapter 3 gives an explicit translation of disposition records into kernel histories and proves it in both directions at the level of ledgers: every disposition step becomes an admissible kernel step, and every kernel step of translated shape whose guard holds comes from a disposition step. Under that result the four kernel operators remain the causal basis; transformation, verification, and discharge are guarded compositions of them; the kernel itself enforces freshness and finality; and everything else the lifecycle requires lives in the guard. What the result does not establish is that the kernel performs the computations the guards check. That stronger, transparent reading is stated as a conjecture.

The two refusals agree on the property the whole corpus cares about: a refusal is recorded rather than erased. They differ in scope. Kernel refusal concerns an option and the history continues around it. Disposition refusal concerns a commitment’s entire future, and a later reconsideration must create a distinct successor commitment rather than reopen the refused one.

Three comparative claims

Alongside the construction, the book tests three claims about the calculus’s position, each of which could turn out false. Each is stated with what would settle it.

The first is that the calculus’s nearest formal relative is the theory of event structures with conflict rather than the π -calculus or the lambda calculus. Every disposition world has a labelled event structure underneath it, together with annotations η carrying principals, reasons, witnesses, certificates, versions, and rules (Chapter 2). The claim is settled positively if the admissibility conditions can be expressed as structural constraints on such a structure over a finite alphabet, and negatively, with η as the identified residue, if some condition depends

irreducibly on evidence predicates.

The second concerns refusal. In its defensible form it says that recorded refusal is not represented by the standard failures model of CSP without adding an internal event or instrumentation. A CSP refusal set is a property of behaviour inferred from outside a process; a recorded refusal is authorized, reasoned, and witnessed data inside it. The stronger claim, that no counterpart exists in any form, would need a separation result, and until one is proved the book asserts only the weaker statement. It is settled by an encoding of recorded refusal into instrumented CSP together with a proof that uninstrumented failures equivalence identifies histories that differ only in their recorded refusals.

The third is that making collapse rule-indexed changes the theory of equivalence rather than relocating it. The observable label of a collapse is the pair $\langle k, o \rangle$, so equal payloads under different rules are distinct. The claim is settled by defining equivalence of processes relative to a rule, $P \approx_k Q$, and determining when it is a congruence, which classical equivalences arise as rules, and whether simultaneous observation of one history under several rules yields a result that van Glabbeek's linear-time/branching-time spectrum does not already state. If it yields none, the spectrum contains the substance, and Part VII says so.

One early piece of evidence bears on the first claim. Chapter 21 proves that replay is invariant under exchanging adjacent records of distinct commitments, so that a ledger determines its disposition state only up to a Mazurkiewicz commutation class. The causal order that the ancestry theorems need is therefore the dependence order of a trace, not the raw order of the ledger, and that is exactly the order an event structure makes primary.

What counts as a result

Positive comparative claims are stated relative to Gorla's encodability criteria and the standard of full abstraction; negative claims follow the model of Palamidessi's separation results. Spine theorems are stated with their hypotheses visible. The liveness result in particular holds only under fairness, responsive external dependencies, terminating transformations, and decidable verification, and is stated with all four. Three grades of assertion are kept apart: results proved here; standard results cited and used without re-derivation, marked where load-bearing; and conjectures or programmatic claims, collected in Appendix E.

The book also uses three levels of validation, and they are not interchangeable:

exhaustive finite checks $\subset$ untyped semantic proofs $\subset$ typed universal guarantees.

The laboratory of Chapter 2 enumerates every admissible ledger up to a bound and finds countermodels and specification errors; it proves nothing about larger worlds. The untyped proofs of Parts III, V, and VI establish the behaviour of every operational derivation. The typed metatheory of Part IV establishes that well-formed programs cannot even express several classes of duplicated, abandoned, or misindexed continuation. Each level catches things the next cannot state, and none substitutes for the one above it.

Status of this edition

Drafted in full: Chapters 1 (Computation with a Past), 2 (Disposition Worlds), 3 (The Kernel and the Disposition Layer), 5 (Syntax and Binding), 12 (Labelled Disposition Semantics), 14

(The Type Algebra), 15 (Preservation, Progress, and Protocol Fidelity), 17 through 20 (Part V), 21 (Replay), 24 (Eventual Disposition), and 27 (Typed Disposition Safety), 6 (Concrete Syntax and Unique Parsing), and 16 (Authority as a Modality). The remaining chapters, chiefly the comparative ones, appear as plans stating what each must establish.

Part I

Orientation

Chapter 1

Computation with a Past

1.1 Four commitments of a calculus

A calculus is sometimes presented as syntax followed by reduction rules. That description omits two choices which are equally fundamental. Besides its terms and steps, a calculus fixes an equivalence and an observational boundary. The untyped lambda calculus, for example, begins with variables, abstractions, and applications; advances by beta reduction; compares terms through a selected convertibility or observational relation; and exposes a result through a normal form, a head form, or interaction with a context. Process calculi alter every component. Their terms denote persisting agents, their transitions carry labels, their equivalences compare behaviours, and their observers distinguish actions rather than returned values alone.

The coupling matters. If an observer sees only final values, two histories may be identified even when one passed through a refusal and the other did not. If refusal is itself observable, that identification becomes invalid. If an append-only record survives the computation, equivalence cannot mean literal equality of histories unless no abstraction is permitted. It must instead mean equality under a declared observation rule.

Definition 1.1 (Disposition). *A disposition is an attributable and recordable determination of the admissible future of a commitment. A disposition may advance the commitment, refuse its continuation, or produce a terminal observation.*

This definition separates refusal from failure. A machine failure may prevent the production of any disposition. A refusal is already a result: it identifies a commitment, an authorized source, a reason or predicate, and a record which persists after the refusal has become historical.

1.2 State and history

The running configuration is written

$$C = \langle P, S, A, O, H \rangle.$$

Here P is a process, S is mutable computational state, A maps commitment identities to their current disposition states, O contains observations, and H is an append-only history. The components are kept separate because they answer different questions. The store says what is currently available to computation. The commitment map says which lifecycle rights are live.

The observation set says what has been made public under a collapse rule. The history says how those facts arose.

Two executions may therefore reach extensionally equal stores while remaining semantically different:

$$S_1 = S_2 \quad \text{but} \quad H_1 \neq H_2.$$

This inequality is not administrative residue. If H_1 contains a refusal and H_2 does not, a later observer may legitimately distinguish them.

Definition 1.2 (History extension). *For histories H and H' , write $H \sqsubseteq H'$ when H is a prefix of H' . A transition system is historically monotone when*

$$C \longrightarrow C' \implies H \sqsubseteq H'.$$

Historical monotonicity does not prohibit correction. It prohibits correction by erasure. A later event may compensate for an earlier event, but the compensation is intelligible only because both remain available.

1.3 The lifecycle

A commitment has an identity c , content of type A , and a lifecycle state. At the untyped operational level we use

$$\begin{aligned} d ::= & \text{open}(q) \mid \text{bound}(q, b) \mid \text{transformed}(b, v) \\ & \mid \text{verified}(v, w) \mid \text{verificationFailed}(v, \varphi) \\ & \mid \text{refused}(r) \mid \text{collapsed}(o). \end{aligned}$$

The live states are `open`, `bound`, `transformed`, `verified`, and `verificationFailed`. The terminal states are `refused` and `collapsed`.

Verification has two outcomes and two successor states. A positive result leads to `verified`, carrying a witness. A negative result leads to `verificationFailed`, carrying an authenticated certificate of the failure. The name is chosen with care: nothing has failed to execute. Verification has returned a certified negative result. The two branches are symmetric: successful verification does not collapse the commitment, and failed verification does not refuse it. Verification classifies evidence; an authority disposes of the continuation.

Refusal is permitted from every live state. This is essential. A reviewer may reject a binding, a transformation may be found inadmissible, or verification may have failed. If refusal were available only from `open`, these outcomes would become unrecorded dead ends rather than dispositions.

Let $\preceq_D$ be the least reflexive and transitive relation containing the constructive edges

$$\begin{aligned} & \text{open} \prec_D \text{bound} \prec_D \text{transformed} \prec_D \text{verified} \prec_D \text{collapsed}, \\ & \text{transformed} \prec_D \text{verificationFailed}, \end{aligned}$$

and a refusal edge from each live state to `refused`. From `verificationFailed` the refusal edge is the only one. The two terminal states are incomparable. Consequently, no ordinary lifecycle transition can turn a refusal into a collapse or a collapse into a refusal.

The lifecycle is small enough to tabulate. For each state s , let $\text{Next}(s)$ be the set of actions that may follow it:

$$\begin{aligned} \text{Next}(\text{open}) &= \{\text{bind}, \text{refuse}\}, & \text{Next}(\text{bound}) &= \{\text{transform}, \text{refuse}\}, \\ \text{Next}(\text{transformed}) &= \{\text{verify}, \text{refuse}\}, & \text{Next}(\text{verified}) &= \{\text{collapse}, \text{refuse}\}, \\ \text{Next}(\text{verificationFailed}) &= \{\text{refuse}\}, & \text{Next}(\text{refused}) &= \text{Next}(\text{collapsed}) = \emptyset, \end{aligned}$$

This *refusal completion table* is the single source from which the rest of the book's descriptions of the lifecycle are derived: the operational rules of Chapter 12, the lifecycle step function of Chapter 17, which is the table with the data carried by each state added, and the session type of Chapter 15, which is the table read as a protocol. Every row for a live state contains **refuse**; every terminal row is empty. Those two facts are, in miniature, the statements that refusal is always available and that terminal dispositions are final. The **verify** entry has two outcomes, to **verified** and to **verificationFailed**. A policy may extend the failed row, for instance with a fresh verification attempt or replacement evidence, but nothing is added to it implicitly.

1.4 Records and the step function

Every disposition, and every discharge of an external obligation, leaves a record in the history. The records are

$$\begin{aligned} &\text{Pop}(c, q), & \text{Compensates}(a, c, q, d), & \text{Bind}(a, c, b), & \text{Transform}(c, f, b, v), \\ & & \text{Verify}(a, c, v, w), & \text{VerificationFailed}(a, c, v, \varphi), \\ \text{Refuse}(a, c, r), & \text{Collapse}(a, c, k, w, o), & \text{Discharged}(a, c, \omega, \varphi), & \text{TimedOut}(a, c, \omega, t, \varphi), \end{aligned}$$

where a names a principal, q a request, b and v values, f a transformation, w a witness, k an observation rule, and o an observation.

A verifier g applied to a commitment and value returns one of two results: $\text{ok}(w)$, a witness, or $\text{fail}(\varphi)$, a *failure certificate*. Certificates have a fixed shape,

$$\varphi = \langle \kappa, g, c, h(v), \text{outcome}, \sigma \rangle,$$

naming the verifier's semantic version κ , the issuer g , the commitment c , a hash $h(v)$ of the value or claim the certificate is about, the outcome it attests, and a signature σ over all of these. The same shape serves for external evidence of discharge, below, with the obligation in place of the value. A negative verification is recorded by its own record, $\text{VerificationFailed}(a, c, v, \varphi)$, carrying the certificate. Refusal reasons r are always written by the refusing principal; there is no reserved reason, because a failed verification is not a refusal.

The *subject* of a record is the commitment whose state it changes. For a compensation record it is the new successor c ; the predecessor d is referred to but not changed. In every other record it is the commitment named first after the principal, or first overall for **Pop** and **Transform**. For a history H , the *projection* $H|_c$ is the subsequence of records whose subject is c , in ledger order.

Two predicates on evidence are fixed: $\text{valid}(w, c, v)$, which says w is a correct witness for c having value v , and

$$\text{checkCert}(\varphi, c, x, \text{outcome}),$$

which says that φ is well formed, that its commitment field is c , its claim field is $h(x)$, its outcome field is the one named, and its signature verifies for the issuer and version it names. The check reads the certificate; it never reruns the verifier. Rerunning may be impossible, expensive, nondeterministic, or altered by a later version, and replay must remain deterministic when the original verifier is gone.

Definition 1.3 (Lifecycle step). *The partial function δ takes a state, or $\perp$ for “not yet introduced”, and a record, and returns the resulting state:*

$$\begin{aligned}
\delta(\perp, \text{Pop}(c, q)) &= \text{open}(q), \\
\delta(\perp, \text{Compensates}(a, c, q, d)) &= \text{open}(q), \\
\delta(\text{open}(q), \text{Bind}(a, c, b)) &= \text{bound}(q, b), \\
\delta(\text{bound}(q, b), \text{Transform}(c, f, b, v)) &= \text{transformed}(b, v), \\
\delta(\text{transformed}(b, v), \text{Verify}(a, c, v, w)) &= \text{verified}(v, w) && \text{if } \text{valid}(w, c, v), \\
\delta(d, \text{Refuse}(a, c, r)) &= \text{refused}(r) && \text{if } \text{live}(d), \\
\delta(\text{transformed}(b, v), \text{VF}(a, c, v, \varphi)) &= \text{vFailed}(v, \varphi) && \text{if } \text{checkCert}(\varphi, c, v, \text{fail}), \\
\delta(d, \text{Discharged}(a, c, \omega, \varphi)) &= d && \text{if } \text{live}(d) \text{ and} \\
&&& \text{checkCert}(\varphi, c, \omega, \text{discharged}), \\
\delta(d, \text{TimedOut}(a, c, \omega, t, \varphi)) &= d && \text{if } \text{live}(d) \text{ and} \\
&&& \text{checkCert}(\varphi, c, \langle \omega, t \rangle, \text{expired}), \\
\delta(\text{verified}(v, w), \text{Collapse}(a, c, k, w, o)) &= \text{collapsed}(o) && \text{if } o = \text{observe}_k(v),
\end{aligned}$$

where **VF** abbreviates the record **VerificationFailed** and **vFailed** the state **verificationFailed**, and δ is undefined in every other case. Repeated variables must match: the b of a transformation record must be the b held in the bound state, the v of a verification record the v produced by transformation, and the w of a collapse record the witness held in the verified state. Write δ^* for the left fold of δ over a sequence from $\perp$.

The step function is the refusal completion table with the data carried by each state checked against the data in each record. Discharge and timeout records are the two clauses that leave the lifecycle state unchanged. A discharge records that principal a discharged an external obligation ω that c was waiting on, with authenticated evidence φ . A timeout records, with evidence from a clock authority a , that ω was *not* discharged within the declared interval t . Neither ends the commitment; both are consumed and checked by the fold, not skipped. Formally δ also carries, beside each state, the set of obligations already settled for c , by discharge or by timeout, and rejects a second settlement of the same obligation; since that set never affects any other clause, it is suppressed from the notation. Note the verification-failure clause: it is admissible only from **transformed**, only with an authentic certificate for that commitment and value, and it leads to a live state, not a terminal one. Refusal is admissible from every live state, **verificationFailed** included, and only by explicit act.

Hypothesis 1.4 (Stability). *The predicates **valid** and **checkCert** and the maps observe_k are deterministic and fixed for the duration of an execution, relative to the verifier versions recorded in witnesses and certificates.*

Stability is what lets a verification carried out at one moment count as evidence at a later moment, and what lets replay re-check a record long after it was appended. Versions matter

because a verifier upgraded mid-execution changes what valid means; recording κ keeps the question “valid under which verifier?” answerable.

Hypothesis 1.5 (Verifier soundness). *If $g(c, v) \Downarrow \text{ok}(w)$ then $\text{valid}(w, c, v)$, and if $g(c, v) \Downarrow \text{fail}(\varphi)$ then $\text{checkCert}(\varphi, c, v, \text{fail})$. Moreover checkCert holds only for certificates actually issued by the issuer named in φ , at the version named, on the commitment and claim named.*

The first half is a requirement on verifiers. The second is an authentication assumption, met in practice by signing certificates with a key held by the verifier.

1.5 A first example

Consider a request q that introduces a fresh commitment c . A value b is bound to it by principal a , a function f transforms b into v , a verifier a' produces witness w , and a'' collapses the result under observation rule k . The successful history is

$$H = \langle \text{Pop}(c, q), \text{Bind}(a, c, b), \text{Transform}(c, f, b, v), \\ \text{Verify}(a', c, v, w), \text{Collapse}(a'', c, k, w, o) \rangle.$$

The identity c threads through the record. It is not replaced by b or v . The witness is indexed by c and v , preventing evidence produced for another commitment from being replayed here.

If verification instead rejects v , the history continues

$$\text{VerificationFailed}(a', c, v, \varphi); \text{Refuse}(h, c, r),$$

where φ is the verifier’s authenticated failure certificate and h is a principal with authority to refuse. The verifier changed the evidentiary state of the commitment; the refusal is a separate act by someone entitled to dispose of it. This is not a truncated successful history. It is a complete refused history, and it retains the evidence of failure beside the decision that followed it. Any later reconsideration must introduce a successor commitment c' and record that c' compensates for the practical matter associated with c .

Chapter 2

Disposition Worlds

A formal semantics becomes usable when a reader can build small models of it by hand, ask whether statements are true in them, and find the smallest model in which a plausible statement fails. This chapter introduces the finite models used throughout the book, together with three instruments for working with them: countermodels, history twins, and causal shuffles. They are elementary enough to use before any metatheory is available, and later chapters show that they were instantiating the central theorems all along. A computation is a finite causal world; a ledger is one serialization of that world; and a collapse rule declares what an observer retains.

2.1 Worlds and configurations

Definition 2.1 (Disposition world). *A disposition world is a tuple*

$$W = \langle E, \prec, \#, \lambda, \mathcal{P}, \mathcal{K} \rangle$$

where E is a finite set of events; $\prec$ is a strict partial order on E (causal precedence); $\#$ is a symmetric, irreflexive relation on E (incompatibility) that is inherited upward, so that $e \# e' \prec e''$ implies $e \# e''$; λ labels each event with a ledger record; $\mathcal{P}$ is an authority policy, a set of triples (a, κ, c) stating that principal a may perform acts of kind κ on commitment c ; and $\mathcal{K}$ is a finite set of observation rules. The commitments of W are the subjects of its labels.

A *configuration* of W is a set $x \subseteq E$ that is downward closed under $\prec$ and contains no two incompatible events. It is one possible state of the computation: the events that have happened. A *linearization* of x is a listing of its events consistent with $\prec$, read as a ledger through λ . A *pointed world* (W, x) fixes a current configuration.

Definition 2.2 (Admissible world). *W is admissible when*

- (W1) *any two events whose records are dependent, in the sense of Definition 21.9, are either $\prec$ -comparable or incompatible;*
- (W2) *every linearization of every configuration has a defined replay;*
- (W3) *for every event labelled with an attributed record, the required authority of Chapter 12 is in $\mathcal{P}$.*

Condition (W1) says the world orders everything that replay cares about. Condition (W2) says every possible state of the world is a legal disposition state. Condition (W3) says nothing in the world was done without entitlement.

Worlds of this kind are prime event structures, in the sense of Nielsen, Plotkin, and Winskel, with events labelled by records and an authority policy attached. The book's first comparative claim is that this is not an accident, and this chapter is where the resemblance first becomes concrete.

A world with empty $\#$ has a single maximal configuration and describes one execution. Every execution produces one:

Proposition 2.3 (Executions are worlds). *Let H be the history of an execution from the empty configuration. The world whose events are the records of H , ordered by the dependence order $\prec_H$ of Definition 21.9, with empty incompatibility and the execution's policy, is admissible.*

Proof. (W1) holds because dependent records are $\prec_H$ -ordered by definition. For (W2), let x be a downset of $\prec_H$. Some linear extension of $\prec_H$ lists x first; it is trace equivalent to H , so by Proposition 21.10 its replay is defined, and replay of a prefix of a defined fold is defined. Any other linearization of x is trace equivalent to that prefix (Theorem 2.9). (W3) is Theorem 12.9. $\square$

What an ordinary event structure lacks

A disposition world has a plain labelled event structure underneath it. Write it as

$$\mathcal{E}_D = \langle E, \leq, \#, \lambda, \eta \rangle,$$

where $\langle E, \leq, \#, \lambda \rangle$ is a labelled prime event structure whose labels $\lambda(e)$ record only the kind of act and its subject, and η carries the disposition annotations: principal, reason, witness, certificate, verifier version, and observation rule. The underlying structure determines configurations and linearizations, and with them everything in the Causal Shuffle theorem below. What it does not determine is admissibility: conditions (W2) and (W3) consult η through valid , checkCert , observe_k , and the policy.

This makes the first comparative claim precise. It is settled in the positive if (W2) and (W3) can be expressed as structural constraints on a labelled event structure over a finite alphabet, in which case the calculus is an event-structure discipline with rich labels. It is settled in the negative if some admissibility condition depends irreducibly on evidence predicates over unbounded data, in which case η is the residue the claim predicts.

2.2 A small language

Formulas are built from atomic statements about a pointed world, the Boolean connectives, quantification over the finitely many commitments, principals, and values of W , and one modality.

- $\text{Live}(c)$, $\text{Refused}(c)$, $\text{Collapsed}(c)$, and the other state predicates hold at (W, x) when replay of a linearization of x puts c in the named state.
- $\text{Occurred}(\rho)$ holds when some event of x is labelled ρ .

- $\text{Authorized}(a, \kappa, c)$ holds when $(a, \kappa, c) \in \mathcal{P}$.
- $\text{Valid}(w, c, v)$ is the validity predicate.
- $\Diamond\varphi$ holds at (W, x) when φ holds at (W, y) for some configuration $y \supseteq x$. $\Box$ is its dual.

The state predicates are well defined only because every linearization of x gives the same replay. That is the Causal Shuffle theorem below, so the language rests on it from its first clause.

Proposition 2.4 (Modal terminal exclusivity). *In every admissible pointed world,*

$$(W, x) \models \text{Refused}(c) \rightarrow \neg\Diamond\text{Collapsed}(c).$$

Proof. Suppose c is refused at x and $y \supseteq x$. The refusal event of c is in y . In any linearization of y , the projection onto c contains a refusal record, and since δ is undefined on every record whose incoming state is terminal, that refusal is the last record of the projection and the fold yields **refused**. So c is not collapsed at y . $\square$

2.3 Countermodels

The most useful operation on worlds is the search for the smallest one that falsifies a proposed statement:

$\text{countermodel}(\varphi, n) =$ a minimal admissible W with at most n events and $W \not\models \varphi$.

Minimality is lexicographic: fewest commitments, then fewest events, then fewest principals, then least causal depth. Small countermodels teach more than arbitrary failing cases, because every event in a minimal countermodel is doing work.

Two examples, found by the laboratory of Section 2.9 under a policy in which a_1 may bind and a_2 may verify:

- The conjecture “if c is verified, the principal who bound c is authorized to verify it” fails in a four-event world, $\text{Pop}(c); \text{Bind}(a_1, c); \text{Transform}(c); \text{Verify}(a_2, c)$. The conjecture was underspecified: authority attaches to acts, not to commitments, and different acts on one commitment may belong to different principals.
- The conjecture “a refused commitment was never verified” fails in a five-event world in which a reviewer refuses after a successful verification. Refusal is available from **verified** as from every live state, and a valid witness does not oblige anyone to accept.
- The conjecture $\text{Live}(c) \rightarrow \Diamond\text{Collapsed}(c)$, that every live commitment can still collapse, fails in a four-event world, $\text{Pop}(c); \text{Bind}(a_1, c); \text{Transform}(c); \text{VF}(a_2, c)$. A commitment whose verification failed is live, but no continuation leads to collapse; the only way out is refusal. This countermodel appeared when the laboratory was rerun after the **verificationFailed** state was introduced, and it is correct: it is the formal content of the decision that a negative verification does not dispose, and that nothing but an authorized refusal disposes after it.

Conversely, no countermodel with at most six events exists for Proposition 2.4, nor for the weaker reachability statement $\text{Live}(c) \rightarrow \Diamond\text{Terminal}(c)$ under that policy.

Remark 2.5 (Horizons). A bounded search for $\Diamond\varphi$ must look far enough ahead. A search that extends worlds only to the global bound will report $\text{Pop}(c_1); \text{Bind}(a_1, c_1); \text{Pop}(c_2)$ as a countermodel to $\text{Live}(c) \rightarrow \Diamond \text{Terminal}(c)$ at bound six, simply because the remaining budget is too small for c_1 to finish. The laboratory therefore extends each world by the longest lifecycle path, four events, beyond its own length. The general moral is that a bounded countermodel to a modal statement is evidence only relative to its horizon.

The same care applies to what the positive result means. The modality $\Diamond$ is existential, so the absence of a countermodel to $\text{Live}(c) \rightarrow \Diamond \text{Terminal}(c)$ is a *reachability* result: from every live state some continuation of at most four records reaches a terminal state, and when any principal holds refusal authority, one record suffices. It is not the liveness statement that every execution eventually disposes of c . Unrelated events may postpone c indefinitely and external obligations may never be answered; eventual disposition is a conditional temporal theorem, stated with its hypotheses in Chapter 24.

2.4 History twins and collapse lenses

A *lens* is an observation rule k applied to a pointed world, yielding some value $k(W, x)$. Two pointed worlds are equivalent under k , written $(W, x) \approx_k (W', x')$, when their lens values agree.

Definition 2.6 (History twins). *Pointed worlds are twins for lenses (k_1, k_2) when they are equivalent under k_1 and not under k_2 .*

The laboratory finds three canonical twins.

1. *Same output, different refusals.* A single commitment that is bound, transformed, verified, and collapsed to o ; and a world in which the same happens while a second commitment is opened and refused. A lens reporting the observed values sees $\{o\}$ in both. A lens reporting refusals distinguishes them.
2. *Same output, different authority.* The same five-event history twice, once collapsed by a_1 and once by a_2 . The value lens cannot tell them apart; a lens reporting who performed the decisive act can.
3. *Same state, different refusal.* The worlds

$$\text{Pop}(c); \text{Refuse}(a_1, c, r) \quad \text{and} \quad \text{Pop}(c); \text{Bind}(a_1, c); \text{Transform}(c); \text{VF}(a_2, c); \text{Refuse}(a_1, c, r).$$

Both leave c refused. One refusal was a decision before any work was done; the other was a decision taken after a certified verification failure, and the ledger keeps the failure beside the decision. This is the operational content of indexing the refusal type by the lifecycle stage, $\text{Refusal}\langle c, A, s \rangle$, in Chapter 14.

Two worlds are never simply equivalent. They are equivalent under a declared lens.

Definition 2.7 (Lens refinement). *Lens k_1 refines k_2 , written $k_1 \preceq k_2$, when $k_2 = h \circ k_1$ for some function h : everything k_2 reports can be computed from what k_1 reports.*

Proposition 2.8. *Refinement is a preorder, and if $k_1 \preceq k_2$ then $(W, x) \approx_{k_1} (W', x')$ implies $(W, x) \approx_{k_2} (W', x')$.*

Proof. Reflexivity takes h to be the identity and transitivity composes the two functions. If k_1 values agree, applying h to both gives equal k_2 values. $\square$

Lenses identified up to mutual refinement correspond to the equivalence relations they induce on pointed worlds, and equivalence relations on a set form a complete lattice under inclusion. The collapse rules of the calculus therefore form a structured spectrum rather than an unrelated collection. Whether that spectrum coincides with van Glabbeek's for the classical equivalences, when those are recast as lenses, is the question of Chapter 25.

2.5 Causal shuffles

The third instrument enumerates every ledger that could have recorded one world: all linearizations of a configuration.

Theorem 2.9 (Causal shuffle). *In a world satisfying (W1), all linearizations of a configuration have the same replay: either all are undefined, or all are defined and equal. In particular this holds in every admissible world.*

Proof. Any two linear extensions of a finite partial order are connected by a sequence of exchanges of adjacent elements that are incomparable in the order: move the element that comes first in the target order to the front by successive adjacent exchanges, each of which passes an element that the target places later and which therefore cannot precede it in the order, and recurse on the rest. Two incomparable events in a configuration are not incompatible, so by (W1) their records are independent. By Proposition 21.10, exchanging adjacent independent records does not change replay, and does not change whether it is defined. The proof uses only (W1), which is why it can be invoked in establishing (W2) for Proposition 2.3. $\square$

The theorem identifies exactly when the sequence numbers a sequencer assigns are incidental. Every order among independent records is one serialization of the same world. Order is semantically necessary only between dependent records, and (W1) requires the world to state it. For the finite class $\mathcal{H}_{\leq 7}$ of Section 2.9, the laboratory confirms the theorem instance by instance: each of the 372,961 linearizations of the 15,167 ledgers in the class replays to the same state as the ledger it came from.

2.6 The witness mixer

Let c_1 and c_2 both be $\text{transformed}(b, v)$, with witnesses w_1 produced for (c_1, v) and w_2 produced for (c_2, v) . Try verifying c_1 with w_2 . A check that asked only whether the witness vouches for the value v would accept, since the values coincide. The step function refuses, because it asks for $\text{valid}(w_2, c_1, v)$.

That refusal depends on the verifier, not only on the calculus. Indexing forces the question to be asked about the right commitment, but the predicate must actually distinguish commitments for the answer to differ.

Hypothesis 2.10 (Index sensitivity). $\text{valid}(w, c, v) \wedge \text{valid}(w, c', v)$ *implies* $c = c'$.

Under this hypothesis evidence is non-transferable: a witness valid for one commitment is valid for no other, and the witness ancestry of Corollary 18.3 cannot be satisfied by borrowed evidence. The hypothesis is a requirement on verifiers, typically met by signing or hashing the commitment identity into the witness, and it is stated here because nothing in the operational rules can supply it.

2.7 Perturbations

The laboratory's third mode takes an admissible world and changes exactly one thing. On the five-event successful history, every perturbation but one is caught, and each is caught by a specific clause. A sixth row comes from a third base, a commitment refused after a certified verification failure, whose certificate is replaced by a forged one:

Perturbation	Detected by
unauthorized binder	authority (W3)
foreign witness in verification	δ : $\text{valid}(w, c, v)$ fails
collapse citing a different witness	δ : witness mismatch
exchange of two dependent records	δ : wrong incoming state
reuse of a commitment identity	δ : identity not fresh
forged failure certificate	δ : checkCert fails
deletion of an interior record	δ : wrong incoming state, <i>usually</i>
deletion of the final record	<i>not detected</i>

The last row is not a flaw in the checker. A ledger with its final record removed is a prefix of an admissible ledger, and every prefix of an admissible ledger is admissible, since the world it describes is simply an earlier configuration. Replay certifies that what is present could have happened. It cannot certify that nothing is missing from the end.

This is an impossibility, not an oversight, and Chapter 21 states it as the Prefix Indistinguishability theorem together with the remedy: a signed checkpoint retained outside the ledger. Completeness itself requires a witness outside the history whose completeness is in question.

The qualification in the interior row is a finding of the second base. Deleting the transformation from

$$\text{Pop}(c) ; \text{Bind}(a_1, c) ; \text{Transform}(c) ; \text{Refuse}(a_1, c, r)$$

leaves $\text{Pop}(c) ; \text{Bind}(a_1, c) ; \text{Refuse}(a_1, c, r)$, which is admissible, because explicit refusal is legal from the bound state as well. The same generosity that makes refusal available at every live stage makes some interior deletions invisible to replay. Any deletion whose remainder is still a legal path escapes detection, and the same anchor is the remedy: a hash chain detects every modification, not only truncation.

2.8 Disposition control

The instruments so far evaluate statements in given worlds. A different question asks which decisions determine a disposition: what must be fixed so that a commitment is guaranteed to end one way.

Definition 2.11 (Interventions, forcing, influence). *An intervention K is a set of constraints on decision points, each removing some records from the enabled set wherever they would occur:*

for instance “verifications of c succeed” or “principal a never refuses c ”. A completion under K is a maximal admissible ledger all of whose steps respect K . Then

- $\text{force}(K, d)$ holds when every completion under K leaves c in disposition d ;
- $\text{influence}_\mu(K, d) = \Pr_\mu[a \text{ completion under } K \text{ leaves } c \text{ in } d]$, for an explicitly declared distribution μ over schedules, initial states, or worlds.

The two are different kinds of claim. Forcing is a universal statement over completions and is proved or refuted by exhaustion. Influence is a probability and depends on μ ; a high influence is not a guarantee. Without a declared μ , a figure of this kind is not a probability at all: it can only be a coverage figure or a reachability count, and should be reported as one. The distinction is taken from recent work on control of Boolean network models of biological systems, where controlling a feedback vertex set together with the source nodes carries a structural guarantee of reaching a target attractor, while smaller control sets found through network modularity are evaluated by *percent coverage*: the proportion of the target outcome that fixing the control set is shown to reproduce. In that study’s biological models, one variant reached about 87% average coverage with control sets slightly larger than the minimum feedback vertex set, and another, choosing one high-ranked feedback vertex per module, reached about 77% with sets about 63% of that size [19]. Coverage measures partial achievement of a target, and influence as defined here is a frequency over schedules, so the two quantities differ. The lesson carries over all the same: a measure of partial success, however high, is not a guarantee.

For a single commitment under the laboratory’s policy, with μ the uniform scheduler that picks among enabled records with equal probability:

- With no intervention, the influence toward collapse is $1/72$ and toward refusal $71/72$. The asymmetry reflects only that refusal options outnumber advancing ones at every stage under a uniform scheduler, not anything about real systems.
- The unique minimal forcing set for *collapse* has three elements: verifications succeed, and neither principal refuses.
- The unique minimal forcing set for *refusal* has one element: verifications fail.
- The best two-element set that does not force collapse, “neither principal refuses”, reaches influence $1/2$: every remaining completion turns on the verifier.

Forcing refusal is cheap and forcing collapse is expensive, because refusal is available everywhere and collapse only at the end of one path. That asymmetry is structural. It is the formal shadow of the book’s insistence that refusal be available from every live state.

Proposition 2.12 (No cycles, one re-entry). *The lifecycle is acyclic, and each commitment’s projection contains at most one opening, four lifecycle steps, and one discharge per obligation. The only rule enabled by a terminal state is compensation, and it creates a new commitment rather than re-entering the old one. Hence in this calculus an infinite execution is never a cycle in any commitment’s lifecycle; it is an unbounded supply of new commitments, arriving through replication or through chains of compensation.*

Proof. Acyclicity and the bound on lifecycle steps are properties of δ . Discharges are bounded by the obligation set. Compensation is the only rule with a terminality premise, and its

subject is fresh. An infinite execution therefore introduces infinitely many commitments, and introductions come only from pop, whose instances beyond the finitely many in the program text come from replication, and from compensation. $\square$

In the Boolean-network setting, a feedback vertex set matters because cycles are what sustain undesired attractors. Here there are no cycles to break in the basic calculus, and a feedback-set analysis would be ornamental. What plays the corresponding role is the set of re-entry points: replication, and the authority to compensate. Removing compensation authority leaves replication as the only source of unbounded work. If the calculus is extended with reopening, retries, delegation cycles, or mutually dependent verification, genuine cycles appear, and a theorem of the feedback-set kind becomes available: an intervention that meets every live dependency cycle removes cyclic obstructions to disposition. It would still not guarantee eventual disposition, which needs fairness and responsive dependencies as well (Chapter 24).

2.9 The laboratory

The examples in this chapter were produced by a small executable reference, `disposition_lab.py`, which implements the records, the step function, and replay as the book defines them, and enumerates every admissible ledger up to a bound by running a maximally nondeterministic process under a fixed policy. Its results are bounded propositions about a named finite class, not theorems, and they are versioned with the grammar they were computed for.

Let $\mathcal{H}_{\leq 7}$ be the class of admissible ledgers of at most seven records over two commitment identities, two principals, one value, one transformation, one observation rule, one external obligation per commitment with one declared deadline, and the policy stated in the source. For version 4 of the laboratory, whose grammar is the book’s current record grammar, including the `verificationFailed` state, authenticated certificates, and discharge and timeout records, $|\mathcal{H}_{\leq 7}| = 15,167$. The laboratory reports each check as a named property with its class, search depth, normalization, expected result, and, when it fails, a minimized counterexample, so that it serves as an executable index of the proof spine. Every property below holds on every $H \in \mathcal{H}_{\leq 7}$:

Property	Statement checked
P-ReplaySound	δ^* of each projection equals replay’s map (the path lemma)
P-TerminalExclusive	at most one terminal record per commitment, and it is last
P-CertificateAuthentic	every verification failure checks against its state and a verifier
P-FailedVerificationLive	a verification failure leaves its commitment live
P-NoVerifierDisposition	no verification record yields a terminal state
P-TraceInvariant	all 372,961 linearizations replay to the state of their ledger
P-WorldAdmissible	every configuration has a defined replay

In addition, for each of the 15,332 pointed worlds in which some commitment is refused, no admissible extension within the horizon collapses it. The companion module `disposition_parser.py` checks the parser properties of Chapter 6. Two properties named in the proof spine, P-RightsAgree and P-DelegationConservative, need the process and typed layers, which the laboratory does not yet model; they are proved in Chapters 15 and 16 and not checked here.

Earlier versions checked earlier grammars, and their figures apply to those grammars only: version 1 (1,230 ledgers), version 2 with certificates and discharge (6,305), and version 3 with

timeout and automatic refusal on failed verification (14,608). The countermodels, twins, and control figures are unchanged across versions, with one deliberate exception: the countermodel to $\text{Live}(c) \rightarrow \Diamond \text{Collapsed}(c)$, which exists only once a failed verification leaves its commitment live. The laboratory's value is not in the counts. It is in finding the smallest world where an idea goes wrong, which is how the horizon remark, the interior-deletion finding, the forged-certificate row, and that countermodel were found.

Exercises

1. Construct the smallest admissible world in which a commitment is compensated. How many events does it need, and why can none be removed?
2. Find twins for the lenses “set of commitments in a terminal state” and “set of observations”.
3. Give a world violating (W1) in which two linearizations of the same configuration replay differently.
4. Define a lens that distinguishes twin 3 above but not twin 1, and place it in the refinement order relative to the value lens and the refusal lens.
5. Show that without Hypothesis 2.10 there is an admissible world in which c_1 is verified with a witness that was produced for c_2 .

Chapter 3

The Kernel and the Disposition Layer

The disposition calculus uses the names of the Spherepop kernel’s operators. This chapter decides whether that is a formalization of the kernel or a related calculus built above it, by giving an explicit translation

$$\llbracket - \rrbracket : \text{disposition records} \longrightarrow \text{kernel histories}$$

and proving two things about it: that every disposition step translates to an admissible kernel step (forward simulation), and that every kernel step of translated shape whose guard holds comes from a disposition step (backward adequacy). The records and the step function it uses were defined in Section 1.4; the operational rules the guards refer to are stated in Chapter 12, and the corollary relating the two uses consistency, established in Chapter 21. Until both directions are proved, the relation between the layers is a conjecture, and it should be read that way.

3.1 The kernel, restated

A kernel history is an append-only sequence of events. Four causal operators act on histories.

- $\text{Pop}(x)$ commits to the option x among those currently available, removing it from what remains possible.
- $\text{Refuse}_r(X)$ documents that the options in X are inadmissible, with reason r , without removing them from the record of what was considered.
- $\text{Bind}(H_1, H_2)$ couples two histories so that each constrains the other’s continuation, without identifying them.
- $\text{Collapse}_k(H)$ observes a history under a rule k , projecting it onto the quotient k induces.

Three structural rules build the terms these operators act on: sequence $H_1; H_2$, tensor $H_1 \otimes H_2$, which forms a joint context from two histories so that an operator can condition one on the other, and branch, which divides one history into two continuations sharing a prefix. A separate annotation channel carries metadata that participates in no causal reduction.

One feature of the kernel matters more than any other here. Applying Collapse_k computes an observation. It does not by itself make that observation a commitment: a history can be observed under many rules, and none of those observations constrains what happens next unless something commits to it. The corpus’s operating-system specification states this as “collapse is rendering” and keeps rendered views out of the authoritative log. The translation below adopts that reading. Under the alternative reading, in which every kernel collapse is itself logged, one step of the translation is absorbed and nothing else changes; the paragraph on collapse in Section 3.4 says which.

3.2 Kernel histories and admissibility

To state a translation precisely, the kernel needs a semantics of its own, and it should be the most permissive one consistent with the four operators. Fix a universe $\mathcal{X}$ of options. A *kernel history* K is a finite sequence of events

$$\text{Pop}(x), \quad \text{Refuse}_r(X), \quad \text{Bind}(x, y),$$

for options $x, y \in \mathcal{X}$, sets $X \subseteq \mathcal{X}$, and reasons r . Write $\text{Popped}(K)$ for the options popped in K and $\text{Refused}(K)$ for the union of the refused sets. An event is *kernel admissible* after K when:

- $\text{Pop}(x)$: $x \notin \text{Popped}(K) \cup \text{Refused}(K)$, so an option can be committed to at most once, and never after it has been documented as inadmissible;
- $\text{Refuse}_r(X)$: always, since documenting inadmissibility removes nothing;
- $\text{Bind}(x, y)$: $x, y \in \text{Popped}(K)$, so only things that exist can be coupled.

A kernel history is admissible when each event is admissible after its prefix. The kernel’s derived choice, $\text{Choice}(x, X) = \text{Pop}(x) \parallel \text{Refuse}(X \setminus \{x\})$, commits to one option and documents the rest as inadmissible. Kernel Collapse_k is a function on histories, a rendering, and not an event.

This semantics enforces very little, and that is deliberate. It says nothing about the order in which options may be popped, nothing about what data they carry, and nothing about who may pop them. Whatever the disposition calculus requires beyond freshness and the permanence of refusal must therefore be supplied by the translation, and the question of this chapter is exactly how much that is.

3.3 Continuation spaces and guards

The options of the translation are the records themselves. A record popped in the kernel is a commitment to that record having occurred.

Definition 3.1 (Continuation space). *For a ledger H and commitment c , the admissible continuation space is*

$$\Omega_c(H) = \{\tau \mid \text{every record of } \tau \text{ has subject } c \text{ and } \delta^*(H|_c \cdot \tau) \text{ is defined}\},$$

the set of futures that c ’s own lifecycle still admits. Its one-step part, the records that could come next, is $\Omega_c^1(H) = \{\rho \mid \langle \rho \rangle \in \Omega_c(H)\}$. Write $\text{Opt}_c(H)$ for the set of all records with subject c not yet popped.

For a live commitment $\Omega_c(H)$ contains the continuations its state, the data it carries, and the evidence predicates allow. For a terminal commitment it contains only the empty continuation.

Definition 3.2 (Guard). *The guard of a record ρ at ledger H , written $G_\rho(H)$, is the conjunction of the premises of the operational rule that appends ρ , read against the state that replay of H reconstructs: the lifecycle and data conditions ($\rho \in \Omega_{\text{subj}(\rho)}^1(H)$), the authority premise, the evaluation or verification premise where there is one ($f \downarrow v$, $g(c, v) \downarrow \text{ok}(w)$ or $\text{fail}(\varphi)$), and for compensation the terminality of the predecessor.*

The guard is a statement in the metalanguage. It is not a kernel operator, and the translation does not emit an event for it. A step whose guard holds is translated; a step whose guard fails is simply not a disposition step. If an implementation wishes to record a failed attempt, it may do so as a kernel refusal of the proposed option or on the annotation channel, but nothing in what follows depends on that choice.

3.4 The translation

Definition 3.3 (Record translation). *For a record ρ with subject c whose guard $G_\rho(H)$ holds, let ℓ_c be the last record of $H|_c$, and define $\llbracket \rho \rrbracket_H$ by*

$$\begin{aligned} \llbracket \text{Pop}(c, q) \rrbracket_H &= \text{Pop}(\rho), \\ \llbracket \text{Compensates}(a, c, q, d) \rrbracket_H &= \text{Pop}(\rho); \text{Bind}(\rho, \ell_d) \quad \text{over } H_c \otimes H_d, \\ \llbracket \rho \rrbracket_H &= \text{Pop}(\rho); \text{Bind}(\rho, \ell_c) \quad \text{for every other non-terminal } \rho, \\ \llbracket \text{Refuse}(a, c, \varrho) \rrbracket_H &= \text{Pop}(\rho) \parallel \text{Refuse}_{(a, \varrho)}(\text{Opt}_c(H) \setminus \{\rho\}); \text{Bind}(\rho, \ell_c), \\ \llbracket \text{Collapse}(a, c, k, w, o) \rrbracket_H &= \text{Pop}(\rho) \parallel \text{Refuse}(\text{Opt}_c(H) \setminus \{\rho\}); \text{Bind}(\rho, \ell_c), \quad o = \text{Collapse}_k(v). \end{aligned}$$

The other non-terminal records are the advancing records (Bind, Transform, Verify) and the discharge and timeout records. The translation of a ledger is $\llbracket \varepsilon \rrbracket = \varepsilon$ and $\llbracket H \cdot \rho \rrbracket = \llbracket H \rrbracket; \llbracket \rho \rrbracket_H$, defined when every guard holds at its prefix. Conversely, the decoding of a kernel history lists its popped records in order.

The kernel history component H_c of a commitment is the chain of its popped records linked by Bind: each record of c is coupled to the one before it, without being identified with it. The clauses fall into four groups.

Openings. Introducing a commitment pops its opening record. Freshness needs nothing extra: a popped option can never be popped again. Compensation also binds the new opening to the predecessor's last record, which needs the tensor rule to form the joint context of two histories, the same requirement that waiting on a child process and mounting a namespace place on the operating-system specification. Bind is the right operator because it couples without identifying, the property developed at length for the kernel's binding in [29]: the successor depends on the predecessor and is a different commitment.

Advancing, discharge, and timeout. Binding, transformation, verification, discharge, and timeout all have the same shape, $\text{Bind} \circ \text{Pop}$ onto the commitment's history. They differ

in payload and in guard. Nothing about transformation or verification requires a kernel operator the other two do not; their distinctive content is in the conditions under which they are admissible.

Terminal steps. Refusal and collapse are both instances of the kernel's derived choice: one option is popped and every other option of c is documented as inadmissible. What distinguishes them is what is popped. A refusal pops an attributed, reasoned decision, and the refused set carries that principal and reason. A collapse pops a committed observation $o = \text{Collapse}_k(v)$: kernel collapse renders an observation, and the pop makes it a commitment. Disposition refusal is therefore kernel refusal of the *whole remaining option space* of c , which is what distinguishes it in scope from kernel refusal of a single option; and disposition collapse is $\text{Pop} \circ \text{Collapse}_k$, a committed observation, rather than kernel collapse alone.

Where the discipline lives. Because the terminal translations refuse every remaining option of c , the kernel's own admissibility enforces finality: after either terminal step, no further record of c can be popped. Freshness is likewise enforced by the kernel. Everything else, the order of the lifecycle, the matching of data between records, the validity of evidence, and authority, is enforced only by the guard. That is the precise content of calling the disposition calculus a discipline over the kernel.

3.5 Forward simulation and backward adequacy

Lemma 3.4 (Records occur once). *If $\text{replay}(H)$ is defined, no record occurs twice in H .*

Proof. A repeated record has the same subject as its first occurrence, so both lie in one projection. Every clause of δ other than discharge and timeout moves strictly along the acyclic lifecycle or out of $\perp$, so the second occurrence would meet a state from which its clause is undefined. A repeated discharge or timeout of the same obligation is rejected by the set of settled obligations carried beside the state. $\square$

Theorem 3.5 (Forward simulation). *If $\text{replay}(H)$ is defined and $G_\rho(H)$ holds, then $\llbracket H \rrbracket; \llbracket \rho \rrbracket_H$ is a kernel-admissible history whose decoding is $H \cdot \rho$.*

Proof. By induction on H the prefix $\llbracket H \rrbracket$ is admissible and decodes to H . For the new events: ρ has not been popped, by Lemma 3.4 applied to $H \cdot \rho$, whose replay is defined because the guard includes $\rho \in \Omega_c^1(H)$. It has not been refused, because the only refusals of options with subject c are those emitted by a terminal step of c , and a terminal c has an empty one-step space, contradicting the guard. So $\text{Pop}(\rho)$ is admissible. Each **Bind** couples ρ to a record already popped. Each **Refuse** is admissible unconditionally. Decoding lists the popped records in order, which is H followed by ρ . $\square$

Corollary 3.6 (Operational forward simulation). *If C is consistent and $C \xrightarrow{\alpha} C'$ appends ρ , then $\llbracket H \rrbracket \xrightarrow{\llbracket \rho \rrbracket_H} \llbracket H' \rrbracket$ in the kernel.*

Proof. The rule that fired has the guard's premises: its lifecycle and data premises give $\rho \in \Omega_c^1(H)$ because replay of H reconstructs A (consistency), and its authority premise is Theorem 12.9. Apply Theorem 3.5. $\square$

The converse direction is where the phrase “discipline over the kernel” earns its meaning. Without it, the translation might embed disposition steps into a more permissive system without the permissiveness showing.

Call a kernel extension β of $\llbracket H \rrbracket$ *typed-guarded* if it has the shape $\llbracket \rho \rrbracket_H$ for some record ρ and $G_\rho(H)$ holds. Let U be the *universal process* over a finite set of names and values: the replication of the sum of every prefix, each followed by $\mathbf{0}$. It can attempt every disposition step, and it performs exactly those whose premises hold.

Theorem 3.7 (Backward adequacy). *Let $C = \langle U, S, A, O, H \rangle$ be consistent, and let $\llbracket H \rrbracket; \beta$ be kernel admissible with β typed-guarded, of shape $\llbracket \rho \rrbracket_H$. Then there is a transition $C \xrightarrow{\alpha} C'$ appending ρ , and $\llbracket H \rrbracket; \beta = \llbracket H' \rrbracket$.*

Proof. The guard is the conjunction of the premises of the rule for ρ , read against replay of H , which by consistency is the runtime state. The universal process offers the corresponding prefix, which by the guard is enabled; firing it appends ρ . The equation is the definition of the translation. $\square$

Together the two theorems say that, under the opaque reading and at the level of ledgers, translated disposition histories are exactly the kernel histories whose steps pass their guards. The disposition calculus adds no causal primitive and removes no kernel history that its guards admit. It is in this sense a *conservative guarded presentation* of the kernel.

Three limits should be kept in view. First, the theorems are relative to the universal process; a particular program offers fewer steps, as every program does, and that is not a failure of adequacy. Second, they hold under the opaque reading, in which evaluation and verification results are inputs checked by the guard. They do not show that the kernel performs those computations. Third, the guard lives in the metalanguage: the kernel does not compute it.

3.6 Opaque and transparent readings

Under the *opaque* reading, the results of evaluation and verification enter as recorded data whose correctness is part of the guard. That is the reading of the theorems above, and it supports every operational and ledger result of the book without any claim about what the kernel can compute.

Under the *transparent* reading, the value v of a transformation is itself produced inside the kernel by its encoding of application, $\text{Collapse}_{\text{subst}}(\text{Bind}(f, b))$, and similarly for verifiers. Only this reading would support the claim that Spherepop itself performs the computation. It requires a theorem the book does not yet have: that the kernel’s application encoding agrees with evaluation in the lambda layer, $\text{Collapse}_{\text{subst}}(\text{Bind}(f, b)) = v$ exactly when $f b \Downarrow v$. It is stated as a conjecture and listed in Appendix E.

3.7 What the translation preserves

Proposition 3.8 (Compositionality). $\llbracket H \cdot H' \rrbracket = \llbracket H \rrbracket; \llbracket H' \rrbracket_H$, where the second translation is taken record by record at the successive prefixes.

Proof. Induction on H' using the defining clause for histories. $\square$

Proposition 3.9 (Locality). *The translation of a record pops and binds only options with its own subject, except that a compensation record also binds to the last record of its predecessor.*

Proof. Inspection of Definition 3.3. □

From these the translation preserves commitment identity (distinct commitments occupy disjoint chains of options, and no clause identifies two of them); causal order (records of one commitment are chained by Bind in ledger order, while independent records touch disjoint options and may be interleaved, so the dependence order of the ledger is carried to the kernel and nothing is added); refusal records (each disposition refusal is a kernel refusal in an append-only history); and declared observations (each (c, k, o) in O corresponds to exactly one popped collapse record carrying k). Finally, the commitment map itself has a kernel reading: δ is an observation rule on histories, and $A = \text{Collapse}_\delta(\llbracket H \rrbracket)$ after decoding. The disposition state is rendered from the history, not stored beside it.

3.8 Decisions fixed by this chapter

1. *No new causal primitive.* Under the opaque reading and at the level of ledgers, the disposition calculus is a conservative guarded presentation of the kernel (Theorems 3.5 and 3.7). Transformation, verification, and discharge are guarded $\text{Bind} \circ \text{Pop}$. Whether the kernel also performs the computations, the transparent reading, is conjectural.
2. *Refusal is available from every live state* and translates to a choice that documents the whole remaining option space of c as inadmissible.
3. *Collapse is always indexed by a rule k* , recorded in the ledger, and means a committed observation, with observable label $\langle k, o \rangle$.
4. *Verification records both outcomes and disposes of neither.* Success appends a `Verify` record and leads to `verified`; failure appends a `VerificationFailed` record carrying an authenticated certificate and leads to the live state `verificationFailed`, from which only an authorized refusal exits.
5. *Plain pop requires no authority.* Introducing an identity commits no principal to anything; the first attributed act on a commitment is its binding, discharge, or refusal.
6. *Replay reconstructs the commitment map and observation set*, not the store. The store is reconstructible exactly when every state-affecting input is recorded.
7. *Discharge and timeout are kernel-derived.* Each is a guarded $\text{Bind} \circ \text{Pop}$ of authenticated evidence onto a waiting commitment's history, not a fifth primitive and not an annotation. Neither ends a commitment.

Chapter 4

Encodings and Their Criteria

Chapter plan. Gorla’s five criteria (compositionality, name invariance, operational correspondence, divergence reflection, success sensitivity) and full abstraction, with examples meeting some and failing others. Three possible outcomes of any comparison: equivalence, strict inclusion, incomparability. Why Turing completeness says almost nothing about expressiveness in the sense needed here.

Part II

Terms and Names

Chapter 5

Syntax and Binding

5.1 Two syntactic categories

The calculus separates expressions from processes. Expressions compute; processes interact and alter disposition. The expression grammar is

$$\begin{aligned} e ::= & x \mid \lambda x:A.e \mid ee \mid \langle e, e \rangle \mid \pi_1 e \mid \pi_2 e \\ & \mid \text{inl } e \mid \text{inr } e \mid \text{case } e \text{ of } \dots . \end{aligned}$$

The process grammar is

$$\begin{aligned} P, Q ::= & \mathbf{0} \mid \text{pop } q \text{ as } c.P \mid \text{pop}_a q \text{ as } c \text{ after } d.P \\ & \mid \text{refuse}_a c \text{ because } r.P \mid \text{bind}_a c \text{ to } b.P \mid \text{transform } c \text{ with } f.P \\ & \mid \text{verify}_a c \text{ using } g \text{ then } P \text{ else } Q \mid \text{collapse}_{a,k} c.P \\ & \mid \text{await}_a c \omega.P \mid \text{await}_a c \omega.P \text{ within}_b t \text{ else } Q \\ & \mid P \mid Q \mid (\nu c)P \mid P + Q \mid !P. \end{aligned}$$

Subscripts name the acting principal a and, for collapse, the observation rule k . Pop and transformation are unattributed: the first only introduces an identity, and the second is a lambda-layer computation. The compensating pop $\text{pop}_a q \text{ as } c \text{ after } d$ introduces a successor c to a terminal commitment d and is attributed, since reconsidering a disposition is itself an act someone must be entitled to perform.

Verification carries two continuations because it may decide either way. A single-continuation form would leave a negative decision without a successor state, and the commitment would sit in transformed with nothing recorded, which is exactly the unrecorded dead end the lifecycle was designed to exclude.

Collapse names no witness. It uses the witness recorded in the commitment's verified state, so a program cannot cite the wrong one, and a well-typed program cannot stall on a mismatch between the witness it holds and the one the state holds. The prefix $\text{await}_a c \omega.P$ suspends the process until principal a discharges an external obligation ω on which commitment c depends, such as a committee decision or a response from an outside service. It is the calculus's positive form of waiting: a process that is waiting says so, names the commitment, and names who it is waiting for, rather than simply failing to move. Waiting itself writes nothing to the ledger; only a successful discharge does. The deadline form $\text{await}_a c \omega.P \text{ within}_b t \text{ else } Q$ adds an explicit

branch: either a discharges ω and the process continues as P , or a clock authority b certifies that the interval t passed without discharge and the process continues as Q . A timeout does not convert silence into refusal. It records that the obligation went unanswered, and it is up to an authorized rule in Q to decide whether the commitment then waits further, is compensated, or is refused.

Replication creates new commitments on every unfolding. Because **pop** binds its identity, the law $!P \equiv P \mid !P$ together with α -conversion yields a fresh copy of each generative binder in each copy of P , and the pop rule's freshness premise forces distinct identities when those copies fire. No two instances ever share a commitment.

A negative verification does not refuse the commitment. The failure branch records the verifier's certificate and continues as Q with the commitment live in **verificationFailed**; if the commitment is to end, Q must contain a refusal by a principal entitled to refuse. A verifier therefore changes the evidentiary state of a commitment without ever disposing of it.

The binder in **pop** q as $c.P$ scopes over P , as does the binder of the compensating pop and restriction $(\nu c)P$. Bound variables in lambda abstractions scope over their bodies. Commitment identities and expression variables form disjoint sorts, and they remain sorted even if a concrete implementation represents them with the same strings.

Definition 5.1 (Alpha-equivalence). *Alpha-equivalence, written $M \equiv_\alpha N$, is the least congruence identifying terms which differ only by a consistent capture-avoiding renaming of bound variables or bound commitment identities.*

5.2 Free names and substitution

The free commitment names of representative processes are defined by

$$\begin{aligned}
\text{fn}(\mathbf{0}) &= \emptyset, \\
\text{fn}(P \mid Q) &= \text{fn}(P) \cup \text{fn}(Q), \\
\text{fn}((\nu c)P) &= \text{fn}(P) \setminus \{c\}, \\
\text{fn}(\text{pop } q \text{ as } c.P) &= \text{fn}(q) \cup (\text{fn}(P) \setminus \{c\}), \\
\text{fn}(\text{pop}_a q \text{ as } c \text{ after } d.P) &= \{d\} \cup \text{fn}(q) \cup (\text{fn}(P) \setminus \{c\}), \\
\text{fn}(\text{bind}_a c \text{ to } b.P) &= \{c\} \cup \text{fn}(b) \cup \text{fn}(P), \\
\text{fn}(\text{verify}_a c \text{ using } g \text{ then } P \text{ else } Q) &= \{c\} \cup \text{fn}(g) \cup \text{fn}(P) \cup \text{fn}(Q), \\
\text{fn}(\text{collapse}_{a,k} c.P) &= \{c\} \cup \text{fn}(P), \quad \text{fn}(\text{await}_a c \omega.P) = \{c, \omega\} \cup \text{fn}(P), \\
\text{fn}(\text{await}_a c \omega.P \text{ within}_b t \text{ else } Q) &= \{c, \omega\} \cup \text{fn}(P) \cup \text{fn}(Q).
\end{aligned}$$

Expression substitution $e[v/x]$ and name substitution $P[d/c]$ are capture avoiding. Whenever a substitution would capture a free name, a bound name is first replaced by a fresh one.

Lemma 5.2 (Freshening). *For every finite set of names N and term M , there exists $M' \equiv_\alpha M$ whose bound names are disjoint from N .*

Proof. The syntax tree of M contains finitely many binders. Rename them one at a time using names outside the union of N , the free names of M , and the names already selected. The supply of names is countably infinite, while the excluded set at each step is finite. $\square$

5.3 Structural congruence

Structural congruence identifies processes whose difference is purely administrative:

$$\begin{aligned} P \mid \mathbf{0} &\equiv P, & P \mid Q &\equiv Q \mid P, & (P \mid Q) \mid R &\equiv P \mid (Q \mid R), \\ P + Q &\equiv Q + P, & !P &\equiv P \mid !P, & (\nu c)(\nu d)P &\equiv (\nu d)(\nu c)P, \\ (\nu c)\mathbf{0} &\equiv \mathbf{0}, & (\nu c)(P \mid Q) &\equiv P \mid (\nu c)Q & \text{ if } c \notin \text{fn}(P), \end{aligned}$$

together with α -equivalence. No structural law may delete a recorded event or identify distinct commitment identities. Structural congruence rearranges a process before an event occurs; it does not rewrite the history afterward.

Theorem 5.3 (Equivariance). *For every capture-avoiding permutation of names ρ ,*

$$C \xrightarrow{\alpha} C' \implies \rho(C) \xrightarrow{\rho(\alpha)} \rho(C').$$

Proof. By induction on the transition derivation of Chapter 12. Each primitive rule compares names only for equality, freshness, or membership in a finite support. Permutations preserve all three properties. The contextual cases are immediate from the induction hypothesis, and the congruence case follows from closure of structural congruence under capture-avoiding renaming. $\square$

The theorem is stated for permutations rather than arbitrary renamings because freshness is preserved by bijections and not in general by many-to-one maps. It licenses every later proof to choose fresh names without the result depending on a particular identifier generator.

Chapter 6

Concrete Syntax and Unique Parsing

Semantic disagreements must not hide inside parsing disagreements. If one source text could be read as two processes, a theorem about processes would say nothing definite about programs. This chapter fixes a concrete syntax and proves that it is read one way only. The claim is made at three levels of strength, and they are kept apart: lexical analysis is deterministic; every accepted token sequence has exactly one derivation; and a bounded check confirms that the implementation and printer agree with that theory. The third does not stand in for the second.

6.1 Lexical classes

Tokens fall into disjoint classes, and each class is recognizable from its first character:

Class	Form	Example
commitment names (<i>CN</i>)	#NAME	#c
principals (<i>PR</i>)	@NAME	@a
obligations (<i>OB</i>)	?NAME	?w
observation rules (<i>RL</i>)	%NAME	%alloc
intervals (<i>TM</i>)	~NAME	~t
reasons (<i>RS</i>)	quoted, with backslash escapes	"late"
values, functions, requests (<i>VAL</i>)	lowercase-initial NAME, not a keyword	f
keywords and punctuation	fixed strings	verify,

Here NAME is an ASCII letter or underscore followed by ASCII letters, digits, and underscores. The classes are chosen to make the sorts of the abstract syntax visible in the text: a commitment name cannot stand where a principal is expected, because the two cannot even be the same token.

Lemma 6.1 (Unique tokenization). *Every string has at most one tokenization.*

Proof. After whitespace, the first character determines the class: the five sigils, the quotation mark, a lowercase letter, or a punctuation character, and these are pairwise distinct. Within a class the token is the longest match, which is unique. A lowercase identifier is a keyword

exactly when it appears in the fixed keyword list. A quoted reason ends at the first unescaped quotation mark. $\square$

Failure certificates, witnesses, and ledger records have no concrete syntax at all. They are runtime objects, produced by verifiers and appended by rules, and no production of the grammar below mentions them. That is a fact about the grammar's formation, not the outcome of a test: no source program can contain a certificate, so none can forge one.

6.2 The grammar

The grammar is stratified by precedence, with choice loosest, parallel composition tighter, and prefixes, restriction, and replication tightest:

$$\begin{aligned}
 P &::= S P_1 \quad P_1 ::= + P \mid \varepsilon \\
 S &::= U S_1 \quad S_1 ::= \mid S \mid \varepsilon \\
 U &::= 0 \mid (P) \mid \text{nu } CN . U \mid ! U \mid \text{pop } POP \\
 &\quad \mid \text{bind } [PR] \text{ } CN \text{ to } VAL . U \mid \text{transform } CN \text{ with } VAL . U \\
 &\quad \mid \text{refuse } [PR] \text{ } CN \text{ because } RS . U \mid \text{collapse } [PR, RL] \text{ } CN . U \\
 &\quad \mid \text{verify } [PR] \text{ } CN \text{ using } VAL \text{ then } U \text{ else } U \mid \text{await } [PR] \text{ } CN \text{ OB } AW \\
 POP &::= VAL \text{ as } CN . U \mid [PR] \text{ } VAL \text{ as } CN \text{ after } CN . U \\
 AW &::= . U \mid \text{within } [PR] \text{ } TM \text{ then } U \text{ else } U
 \end{aligned}$$

Choice and parallel composition associate to the right, by the direction of their recursion. Parentheses reset precedence.

One scoping decision needs stating because readers from other traditions may expect otherwise. The continuation after a prefix's dot, and each branch after **then** and **else**, is a unary process U , not an arbitrary process. So

$$\text{pop } q \text{ as } \#c . \quad 0 \mid 0 \quad \text{reads as} \quad (\text{pop } q \text{ as } c.0) \mid 0,$$

and a parallel continuation must be parenthesized, $\text{pop } q \text{ as } \#c . \quad (0 \mid 0)$. The same holds for the branches of verification and of a deadline. This is the usual convention of CCS, where prefix binds more tightly than parallel composition, and it is what makes the two-branch forms unambiguous: every **then** has exactly one **else**, and no **else** can be claimed by more than one **then**. The deadline form is distinguished from the plain **await** by its second token after the obligation, **within** rather than the dot, so there is no dangling-else problem there either.

6.3 Unique derivations

For a nonterminal X and one of its alternatives β , the *predict set* is $\text{FIRST}(\beta)$, together with $\text{FOLLOW}(X)$ if β can derive the empty string. A grammar is LL(1) when, for each nonterminal, the predict sets of its alternatives are pairwise disjoint. For the grammar above:

- P_1 : the alternative $+P$ is predicted by $+$; the empty alternative by $\text{FOLLOW}(P_1) = \text{FOLLOW}(P) = \{\}, \{\$ \}$.

- S_1 : $|$ S is predicted by $|$; the empty alternative by $\text{FOLLOW}(S) = \{+,), \$\}$.
- U : each of the eleven alternatives begins with a distinct terminal.
- POP : VAL versus $[$.
- AW : $.$ versus within .

The laboratory computes the same sets mechanically from the grammar and finds no conflict (property P-ParserLL1).

Theorem 6.2 (Unique parsing). *Every token sequence in the language of the grammar has exactly one derivation. Consequently, by Lemma 6.1, every accepted source text denotes exactly one abstract syntax tree.*

Proof. Suppose a token sequence has two distinct leftmost derivations. Consider the first step at which they differ. Both expand the same leftmost nonterminal X , with different alternatives $\beta \neq \beta'$, and the remaining input begins with the same token t . In each derivation, t is either the first token derived from the chosen alternative, or, if the alternative derives the empty string there, a token that follows X . So t lies in the predict sets of both β and β' , contradicting the LL(1) property. The conversion from derivation to abstract syntax tree is a function, so the tree is unique too. $\square$

Structural congruence plays no part in this theorem. The texts `0 | pop q as #c . 0` and `pop q as #c . 0 | 0` have different, unique trees; that those trees denote congruent processes is a fact about the semantics, not an ambiguity of the syntax.

6.4 Adequacy of the printer

Let $\text{show}(P)$ print a tree with the fewest parentheses its precedence requires, and let $\text{Concrete}(P)$ be the set of texts obtained from $\text{show}(P)$ by wrapping any subterms in additional, redundant parentheses.

Theorem 6.3 (Adequacy). *$\text{parse}(s) = P$ if and only if $s \in \text{Concrete}(P)$.*

Proof. From right to left, by induction on P : the minimal printing parses to P because it inserts parentheses exactly where the precedence stratification requires them, and a redundant pair around a subterm is the production $U \rightarrow (P)$, whose tree is the tree of the enclosed process. From left to right, by induction on the unique derivation of s : the conversion to a tree discards only parentheses and the empty expansions of P_1 and S_1 , so s differs from $\text{show}(\text{parse}(s))$ only by parentheses around subterms, which is membership in $\text{Concrete}(\text{parse}(s))$. $\square$

6.5 The executable check

The reference parser is generated from the same grammar: a table-driven predictive parser whose table is the one computed for the LL(1) check, followed by a conversion from parse trees to abstract syntax trees. There is one grammar in the implementation, not two, so the grammar that is analyzed is the grammar that is parsed.

Beyond the general property, the module checks bounded properties over every abstract syntax tree with at most five constructors, 9,469 distinct trees over a small alphabet of atoms. The alphabet can be small because parsing depends only on token classes, not on which name or value appears. For every such P :

- P-ParserUnique: $\text{show}(P)$ has exactly one derivation, counted by a chart that enumerates all derivations rather than following the table;
- P-ParserRoundTrip: $\text{parse}(\text{show}(P)) = P$, and parse of a fully redundantly parenthesized printing of P is also P ;
- P-ParserNormalize: $\text{show}(\text{parse}(\text{show}(P))) = \text{show}(P)$, and $\text{parse}(\text{show}(\text{parse}(s))) = \text{parse}(s)$ for the redundantly parenthesized s , so redundant parentheses disappear without changing the tree.

Eleven named cases pin down the decisions of this chapter with expected trees, among them prefix scope, the relative precedence of choice and parallel composition, right associativity, the unary branches of verification, the deadline form, and escaped reasons (P-ParserNamed). Seventeen malformed inputs are rejected, among them an unquoted value in the reason position, an unterminated reason, a keyword in a value position, an empty parenthesized process, an extra **else**, a malformed rule index, an obligation in a commitment position, and a deadline lacking its clock principal (P-ParserRejects). As a negative control, adding an ambiguous alternative to the grammar produces both an LL(1) conflict and a sentence with two derivations; the checks detect the ambiguity they are meant to detect.

These checks establish that the implementation and printer agree with the theory over the enumerated class. The universal statement is Theorem 6.2, which rests on the LL(1) property and not on the enumeration.

Chapter 7

The Lambda Layer

Chapter plan. Reduction and big-step evaluation $e \Downarrow v$ for the value language, which the transformation and verification rules consume as a premise. Compares the kernel encodings of abstraction and application with the lambda layer’s own, and asks what a history retains that a normal form discards.

Chapter 8

Combinators, Erasure, and Linear Resource

Chapter plan. Uses combinatory logic to test how much depends on names, and the discarding combinator to ask whether erasure must be recorded. Introduces linear logic as the resource discipline Part IV needs, and compares kernel **Pop**'s removal of an option with linear consumption of a hypothesis.

Part III

Processes and Labelled Transitions

Chapter 9

CCS: Synchronization and Binding

Chapter plan. A finite CCS fragment encoded into disposition histories and the converse examined. The central question is whether binding is synchronization or something weaker, since a CCS handshake consumes both complementary actions in one step. The worked encoding of Appendix D belongs here.

Chapter 10

CSP and the Two Refusals

Chapter plan. Sets failures-semantics refusal sets against recorded refusal. The chapter's target is the weakened claim of the Preface: that recorded refusal is not represented by the standard failures model without an added internal event or instrumentation. It gives the instrumented encoding and then tries to prove the separation, that uninstrumented failures equivalence identifies histories differing only in recorded refusals; the history twins of Chapter 2 are the candidate witnesses. The sequence of a certified verification failure followed by a separately authorized refusal gives a sharp test case: a refusal whose grounds are recorded evidence, for which a refusal set has no slot at all.

Chapter 11

The Pi-Calculus: Restriction and Fresh Commitments

Chapter plan. Name passing, scope extrusion, and restriction. Develops the remark of Chapter 12 that restriction here scopes naming but not visibility, and asks whether this makes the disposition calculus's (νc) closer to a capability discipline than to π -calculus privacy.

Chapter 12

Labelled Disposition Semantics

12.1 Actions and records

Observable actions are

$$\alpha ::= \text{pop}(c, q) \mid \text{compensate}(a, c, q, d) \mid \text{bind}(a, c, b) \mid \text{transform}(c, f, b, v) \\ \mid \text{verify}(a, c, v, w) \mid \text{refuse}(a, c, r) \mid \text{collapse}(a, c, k, w, o).$$

Every observable action has a canonical ledger record $\text{record}(\alpha)$, the record of Section 1.4 with the capitalized constructor and the same fields. This pairing is part of the semantics rather than an optional logging policy. Subjects and projections are as defined there.

12.2 Primitive rules

POP introduces a fresh commitment:

$$\frac{c \notin \text{dom}(A)}{\langle \text{pop } q \text{ as } c.P, S, A, O, H \rangle \xrightarrow{\text{pop}(c, q)} \langle P, S, A[c \mapsto \text{open}(q)], O, H \cdot \text{Pop}(c, q) \rangle}.$$

A compensating pop introduces a fresh successor to a terminal commitment:

$$\frac{c \notin \text{dom}(A) \quad \text{terminal}(A(d)) \quad \text{may}(a, \text{compensate}, d)}{\langle \text{pop}_a q \text{ as } c \text{ after } d.P, S, A, O, H \rangle \xrightarrow{\text{compensate}(a, c, q, d)} \langle P, S, A[c \mapsto \text{open}(q)], O, H \cdot \text{Compensates}(a, c, q, d) \rangle}.$$

Binding advances an open commitment while preserving its identity:

$$\frac{A(c) = \text{open}(q) \quad \text{may}(a, \text{bind}, c)}{\langle \text{bind}_a c \text{ to } b.P, S, A, O, H \rangle \xrightarrow{\text{bind}(a, c, b)} \langle P, S, A[c \mapsto \text{bound}(q, b)], O, H \cdot \text{Bind}(a, c, b) \rangle}.$$

Transformation evaluates a lambda-layer function and records both its input and output:

$$\frac{A(c) = \text{bound}(q, b) \quad f b \Downarrow v}{\langle \text{transform } c \text{ with } f.P, S, A, O, H \rangle \xrightarrow{\text{transform}(c, f, b, v)} \langle P, S, A[c \mapsto \text{transformed}(b, v)], O, H \cdot \text{Transform}(c, f, b, v) \rangle}.$$

Verification runs the verifier and branches on its result. Write

$$V \equiv \text{verify}_a c \text{ using } g \text{ then } P \text{ else } Q.$$

The successful branch produces an indexed witness:

$$\frac{A(c) = \text{transformed}(b, v) \quad \text{may}(a, \text{verify}, c) \quad g(c, v) \Downarrow \text{ok}(w)}{\langle V, S, A, O, H \rangle \xrightarrow{\text{verify}(a, c, v, w)} \langle P, S, A[c \mapsto \text{verified}(v, w)], O, H \cdot \text{Verify}(a, c, v, w) \rangle}.$$

The failing branch records the failure certificate and leaves the commitment live:

$$\frac{A(c) = \text{transformed}(b, v) \quad \text{may}(a, \text{verify}, c) \quad g(c, v) \Downarrow \text{fail}(\varphi)}{\langle V, S, A, O, H \rangle \xrightarrow{\text{verificationFailed}(a, c, v, \varphi)} \langle Q, S, A[c \mapsto \text{verificationFailed}(v, \varphi)], O, H \cdot \text{VerificationFailed}(a, c, v, \varphi) \rangle}.$$

Explicit refusal is available from any live state d :

$$\frac{A(c) = d \quad \text{live}(d) \quad \text{may}(a, \text{refuse}, c)}{\langle \text{refuse}_a c \text{ because } r, P, S, A, O, H \rangle \xrightarrow{\text{refuse}(a, c, r)} \langle P, S, A[c \mapsto \text{refused}(r)], O, H \cdot \text{Refuse}(a, c, r) \rangle}.$$

Collapse commits to an observation of a verified commitment, using the witness held in its state:

$$\frac{A(c) = \text{verified}(v, w) \quad \text{may}(a, \text{collapse}, c) \quad o = \text{observe}_k(v)}{\langle \text{collapse}_{a,k} c.P, S, A, O, H \rangle \xrightarrow{\text{collapse}(a, c, k, w, o)} \left\langle P, S, A[c \mapsto \text{collapsed}(o)], O \cup \{(c, k, o)\}, \right\rangle \left\langle H \cdot \text{Collapse}(a, c, k, w, o) \right\rangle}.$$

The subscript k names the observation rule, and the record carries it. Collapse is therefore not one universal quotient imposed on every history. Different applications may expose different distinctions, but every use declares which rule did the identifying. The observable label of a collapse is the pair $\langle k, o \rangle$, not the payload o alone, so equal payloads produced under different rules remain distinct: $\langle k_1, o \rangle \neq \langle k_2, o \rangle$ when $k_1 \neq k_2$. An observer who wants to ignore the rule may project it away, but that projection is itself a declared collapse rule, not a default. Observations are stored as (c, k, o) , so two commitments collapsing to equal values also remain distinguishable in O .

External obligations are discharged by the environment, with authenticated evidence:

$$\frac{\text{live}(A(c)) \quad \text{may}(a, \text{discharge}, \omega) \quad \text{checkCert}(\varphi, c, \omega, \text{discharged})}{\langle \text{await}_a c \omega.P, S, A, O, H \rangle \xrightarrow{\text{discharge}(a, c, \omega, \varphi)} \langle P, S, A, O, H \cdot \text{Discharged}(a, c, \omega, \varphi) \rangle}.$$

For the deadline form, discharge takes the first branch by the same rule, and a certified timeout takes the second:

$$\frac{\text{live}(A(c)) \quad \text{may}(b, \text{clock}, \omega) \quad \text{checkCert}(\varphi, c, \langle \omega, t \rangle, \text{expired})}{\langle \text{await}_a c \omega.P \text{ within } t \text{ else } Q, S, A, O, H \rangle \xrightarrow{\text{timeout}(b, c, \omega, t, \varphi)} \langle Q, S, A, O, H \cdot \text{TimedOut}(b, c, \omega, t, \varphi) \rangle}.$$

The discharge record must be classified, since the book has just argued that no fifth causal primitive is needed. It is *kernel-derived*: a guarded $\text{Bind} \circ \text{Pop}$ of an evidence-carrying proposal to the history of c , the same shape as binding, transformation, and verification (Chapter 3), differing from them only in leaving the lifecycle state where it was. It is not a primitive and not a mere annotation, because it changes what the process may do next. Three nearby things are *not* ledger events and must stay distinct from it: a pending obligation, which is process state (the await prefix is simply still there); a denied attempt to discharge, which belongs at most on the annotation channel; and nonresponse, which is the absence of any discharge. None of these ends the commitment. If a pending obligation is to end it, an authorized principal must refuse it explicitly. A timeout record is classified the same way as a discharge: kernel-derived, lifecycle-neutral, and authenticated.

The stability hypothesis of Section 1.4 applies to the validity predicate, failure certificates, and observation maps used in these rules.

12.3 Contextual rules

The primitive rules act on a prefix at the top of the process. Parallel components share the store, commitment map, observation set, and history, so a step of one component is a step of the whole:

$$\begin{array}{c}
\frac{\langle P, S, A, O, H \rangle \xrightarrow{\alpha} \langle P', S', A', O', H' \rangle}{\langle P \mid Q, S, A, O, H \rangle \xrightarrow{\alpha} \langle P' \mid Q, S', A', O', H' \rangle} \quad \frac{\langle P, S, A, O, H \rangle \xrightarrow{\alpha} \langle P', S', A', O', H' \rangle}{\langle P + Q, S, A, O, H \rangle \xrightarrow{\alpha} \langle P', S', A', O', H' \rangle} \\
\\
\frac{\langle P, S, A, O, H \rangle \xrightarrow{\alpha} \langle P', S', A', O', H' \rangle}{\langle (\nu c)P, S, A, O, H \rangle \xrightarrow{\alpha} \langle (\nu c)P', S', A', O', H' \rangle} \quad \frac{P \equiv P_1 \quad \langle P_1, \dots \rangle \xrightarrow{\alpha} \langle P_2, \dots \rangle \quad P_2 \equiv P'}{\langle P, \dots \rangle \xrightarrow{\alpha} \langle P', \dots \rangle}
\end{array}$$

Choice is resolved by whichever summand acts first. The unchosen summand is neither evaluated nor refused: the rule for $P + Q$ requires a transition of P only, inspects no premise of Q , and records nothing about Q . Internal choice resolution is therefore a different thing from disposition refusal, and nothing about a discarded branch can later be read off the ledger.

Remark 12.1 (Restriction scopes naming, not visibility). In the π -calculus, restriction makes a name private and hides actions on it from the environment. Here the restriction rule places no side condition on the label, and the record is appended to a history shared by every component. Restriction here is lexical scope, not confidentiality: it limits which parts of a process can *name* a commitment, and so which parts can act on it, but not who can *see* what happened to it. The ledger is not scoped. Whether this makes restriction behave like a capability discipline is a question for Chapter 11, not a claim of this one.

12.4 Worked derivations

The successful route. Let

$$\begin{aligned}
P_0 = & \text{pop } q \text{ as } c. \text{bind}_a c \text{ to } b. \text{transform } c \text{ with } f. \\
& \text{verify}_{a'} c \text{ using } g \text{ then collapse}_{a'',k} c.0 \text{ else } 0,
\end{aligned}$$

with $f b \Downarrow v$, $g(c, v) \Downarrow \text{ok}(w)$, $o = \text{observe}_k(v)$, and each principal holding the authority its step requires. From the empty configuration the five primitive rules fire in order:

$$C_0 \xrightarrow{\text{pop}(c,q)} C_1 \xrightarrow{\text{bind}(a,c,b)} C_2 \xrightarrow{\text{transform}(c,f,b,v)} C_3 \xrightarrow{\text{verify}(a',c,v,w)} C_4 \xrightarrow{\text{collapse}(a'',c,k,w,o)} C_5,$$

with

i	$A_i(c)$	H_i
1	$\text{open}(q)$	$\langle \text{Pop}(c, q) \rangle$
2	$\text{bound}(q, b)$	$H_1 \cdot \text{Bind}(a, c, b)$
3	$\text{transformed}(b, v)$	$H_2 \cdot \text{Transform}(c, f, b, v)$
4	$\text{verified}(v, w)$	$H_3 \cdot \text{Verify}(a', c, v, w)$
5	$\text{collapsed}(o)$	$H_4 \cdot \text{Collapse}(a'', c, k, w, o)$

and $O_5 = \{(c, k, o)\}$. Each history is a proper prefix of the next. The premise of each rule is visible in the previous row: the bind rule needs the **open** state of row 1, the transform rule reads the b of row 2, and so on.

Refusal at every live stage. The four refusal routes are not redundant, because they establish that the refusal rule covers every live state, the failed state included.

1. *After pop.* From C_1 , $\text{refuse}_a c$ because r fires with $A_1(c) = \text{open}(q)$, giving $H_1 \cdot \text{Refuse}(a, c, r)$.
2. *After binding.* From C_2 , a reviewer with refusal authority judges the bound value unsuitable; the explicit rule fires from $\text{bound}(q, b)$.
3. *After an inadmissible transformation.* From C_3 , the explicit rule fires from $\text{transformed}(b, v)$ with a reason stating why v is unacceptable.
4. *After negative verification.* From C_3 , if $g(c, v) \Downarrow \text{fail}(\varphi)$, the failing verification rule fires and the process continues with its *else* branch, giving $H_3 \cdot \text{VerificationFailed}(a', c, v, \varphi)$ with c live in $\text{verificationFailed}(v, \varphi)$. The *else* branch then contains the refusal, $\text{refuse}_h c$ because r , by a principal h holding refusal authority, and the explicit refusal rule fires from the failed state.

Remark 12.2 (Denial of an actor is not refusal of a commitment). One tempting fifth route is “refusal because the binding was unauthorized”. There is no such rule, and there should not be:

$$\neg \text{may}(a, \text{bind}, c) \not\Rightarrow \text{Refused}(c).$$

If an unauthorized attempt refused c , authority failure would become an ambient cancellation capability: any principal could end any commitment by attempting something it is not entitled to do. Three outcomes must be kept apart.

- $\text{DeniedAttempt}(a, c, \alpha)$: a principal tried an act it may not perform. Nothing about c changes. If recorded at all, it belongs on the kernel’s non-causal annotation channel, an audit plane beside the ledger, where it supports security analysis without becoming a causal event.
- $\text{Blocked}(c, \omega)$: the commitment cannot advance until an external obligation ω is discharged. It is a positive runtime form, introduced in Chapter 15, and it changes nothing about the lifecycle either.
- $\text{Refused}(b, c, r)$: an authorized principal b ended the commitment for reason r . Only this outcome changes the lifecycle.

In the present calculus an unauthorized attempt simply has no transition. In the typed calculus it cannot even be written, since every attributed prefix requires its authority as a typing premise.

Independent steps commute. Let $A(c) = \text{open}(q)$ and $A(c') = \text{open}(q')$ with $c \neq c'$, and let the process be $\text{bind}_a c \text{ to } b.0 \mid \text{bind}_{a'} c' \text{ to } b'.0$. Either component may act first, by the parallel rule (with commutativity of $\mid$ for the right component). The two interleavings reach the same process, commitment map, and observation set, and histories

$$H \cdot \text{Bind}(a, c, b) \cdot \text{Bind}(a', c', b') \quad \text{and} \quad H \cdot \text{Bind}(a', c', b') \cdot \text{Bind}(a, c, b),$$

which differ by one exchange of independent records. In general:

Proposition 12.3 (Diamond for independent steps). *Let $C = \langle P_1 \mid P_2, S, A, O, H \rangle$, and suppose P_1 can perform α and P_2 can perform β from C , with $\text{record}(\alpha)$ and $\text{record}(\beta)$ independent in the sense of Definition 21.9. Then both interleavings are possible and reach configurations equal in every component except the history, where they differ by exchanging the two records.*

Proof. The premises of each rule read the commitment map only at the subject of its record, and, for compensation, at its predecessor. By independence, the other step writes neither key (Theorem 12.5), so each premise still holds after the other step, and each update produces the same value in either order. The process components evolve separately. Observation sets are joined by union. A pop and a compensating pop each require their subject to be fresh; independence gives distinct subjects, so neither introduction uses the other's identity. $\square$

The hypothesis that the two steps come from different parallel components matters. In $\text{bind}_a c \text{ to } b. \text{bind}_{a'} c' \text{ to } b'. \mathbf{0}$ the records are still independent, but only one order is possible, because the prefix imposes it. Independence of records says replay does not care about their order; it does not say the program allows both.

Competing terminal steps. Let $A(c) = \text{verified}(v, w)$ and let the process be $\text{refuse}_a c$ because $r. \mathbf{0} \mid \text{collapse}_{a', k} c. \mathbf{0}$. Either component may fire first. If the refusal fires, c becomes $\text{refused}(r)$ and the collapse rule's premise $A(c) = \text{verified}(v, w)$ fails. If the collapse fires, c becomes $\text{collapsed}(o)$, which is not live, and the refusal rule's premise fails. Exactly one terminal record is appended.

This settles a point about concurrency. Terminal exclusivity does not rest on the order in which rules are written; it rests on the commitment map being shared, so that the first terminal step falsifies the premise of every competitor. That in turn rests on an assumption the interleaving semantics makes silently.

Hypothesis 12.4 (Atomic commitment map). *Each transition reads the commitment map, checks its premises, and updates the map as one indivisible step.*

Under this hypothesis exclusivity is *operational*. In a distributed implementation the hypothesis must be earned: without a sequencer, a compare-and-swap, a consensus object, or some other linearization point, two replicas could each read a live state, each append a terminal record, and produce incompatible dispositions that no later replay can reconcile. Chapter 23 states what each mechanism provides. There are therefore two grades of the result: operational exclusivity, which holds for every execution under Hypothesis 12.4, and static exclusion of competitors, which holds for well-typed programs. What the untyped calculus does not do is say anything about the loser, which is left with a prefix it can never perform and no record of why. The linear discipline of Part IV turns that situation into a static error: a well-typed program cannot hold two rights to advance the same commitment, so it cannot contain both components in the first place.

12.5 First preservation results

Theorem 12.5 (Commitment identity). *If $C \xrightarrow{\alpha} C'$, then $\text{dom}(A) \subseteq \text{dom}(A')$, and*

$$\text{dom}(A') \setminus \text{dom}(A) = \begin{cases} \{c\} & \text{if } \alpha = \text{pop}(c, q) \text{ or } \alpha = \text{compensate}(a, c, q, d), \\ \emptyset & \text{otherwise,} \end{cases}$$

with $c \notin \text{dom}(A)$ in the first case. Moreover $A'(e) = A(e)$ for every $e \neq \text{subj}(\text{record}(\alpha))$.

Proof. By induction on the derivation. Each primitive rule updates A at no more than one key, the subject of its record; the discharge and timeout rules write back the value already there. The two introducing rules have $c \notin \text{dom}(A)$ as a premise and add that key; every other primitive rule has a premise $A(c) = \dots$ and so updates an existing key. The contextual rules pass A and A' through unchanged from their premise. $\square$

Since the domain never shrinks, a commitment identity once introduced remains in the map for the rest of the execution. The spine's provision for an archival operation that preserves identity in the ledger is therefore not needed in the present calculus; it becomes relevant only if a later extension removes terminal commitments from A for efficiency.

Theorem 12.6 (Lifecycle monotonicity). *If $C \rightarrow C'$ and c occurs in both commitment maps, then*

$$A(c) \preceq_D A'(c).$$

In particular, no ordinary transition reopens a refused or collapsed commitment.

Proof. By cases on the primitive rule at the root of the derivation, the contextual rules transmitting the property. The introducing rules concern a fresh identifier and, by Theorem 12.5, leave every existing key unchanged; in particular a compensating pop does not alter its predecessor. Each advancing rule follows one generating edge of $\preceq_D$. Both refusing rules follow the refusal edge from a live state. Neither terminal state satisfies the premise of an advancing or refusing rule. $\square$

Theorem 12.7 (History monotonicity). *If $C \xrightarrow{\alpha} C'$, then*

$$H' = H \cdot \text{record}(\alpha),$$

and hence $H \sqsubseteq H'$.

Proof. Inspection of the primitive rules shows that each appends its paired record. The contextual rules pass the history through unchanged from their premise, and structural congruence acts on the process component alone. No rule has a premise or conclusion in which a history suffix is removed or replaced. $\square$

Corollary 12.8 (Terminal exclusivity, map form). *Along any execution, no commitment enters both refused and collapsed.*

Proof. Whichever terminal state is entered first, lifecycle monotonicity keeps the commitment in that state at every later configuration, since $\preceq_D$ relates each terminal state only to itself. By Theorem 12.5 the identity cannot be removed and reintroduced in a different state. $\square$

The history form of this corollary, which is what an auditor reading the ledger needs, is Corollary 19.2.

12.6 Authority

Define the authority an action requires, $\text{auth}(\alpha)$, by

$$\begin{aligned} \text{auth}(\text{bind}(a, c, b)) &= \text{bind}, & \text{auth}(\text{verify}(a, c, v, w)) &= \text{verify}, \\ \text{auth}(\text{collapse}(a, c, k, w, o)) &= \text{collapse}, & \text{auth}(\text{compensate}(a, c, q, d)) &= \text{compensate}, \end{aligned}$$

and

$$\text{auth}(\text{refuse}(a, c, r)) = \text{refuse}, \quad \text{auth}(\text{verificationFailed}(a, c, v, \varphi)) = \text{verify},$$

$\text{auth}(\text{discharge}(a, c, \omega, \varphi)) = \text{discharge}$, and $\text{auth}(\text{timeout}(b, c, \omega, t, \varphi)) = \text{clock}$. The object of the required authority is the action's subject, except for compensation, where it is the predecessor d , and discharge and timeout, where it is the obligation ω .

Theorem 12.9 (Authorized transition). *If $C \xrightarrow{\alpha} C'$ and α is attributed to principal a , then $\text{may}(a, \text{auth}(\alpha), x)$ held in C , where x is the object of the required authority.*

Proof. By inversion on the derivation. The contextual rules preserve the label, so the root primitive rule determines it, and every attributed primitive rule carries the may premise for exactly the kind auth assigns to its label. In particular a refusal label comes only from the explicit refusal rule, whose premise is refusal authority, and a verification-failure label only from the failing verification rule, whose premise is verification authority. $\square$

Two different guarantees concern verification failures, and each has its own scope.

Proposition 12.10 (Source authenticity). *If $\text{VerificationFailed}(a, c, v, \varphi)$ occurs in a history produced by execution, then at the configuration where it was appended, c was $\text{transformed}(b, v)$ for some b , principal a held verification authority over c , and the verifier named in the process returned $\text{fail}(\varphi)$ on (c, v) .*

Proof. By history monotonicity the record was appended by a single transition, which can only have been an instance of the failing verification rule, whose premises are the statement. $\square$

This is a statement about programs. It says nothing about a ledger that arrives from elsewhere, since someone with write access to storage could append a record directly, without running any process. For that setting the guarantee comes from replay.

Proposition 12.11 (Ledger acceptance). *If $\delta^*(H|_c)$ is defined and $H|_c$ contains the record $\text{VerificationFailed}(a, c, v, \varphi)$, then the preceding record of $H|_c$ left c in a state $\text{transformed}(b, v)$ for which*

$$\text{checkCert}(\varphi, c, v, \text{fail})$$

holds. Under Hypothesis 1.5, φ was issued by the verifier it names, at the version it names, on that commitment and value.

Proof. The only clause of δ accepting the record is the verification-failure clause, whose incoming state and side condition are the statement. The second sentence is the authentication half of Hypothesis 1.5. $\square$

What replay cannot check is that the principal a in the record held verification authority; that requires the policy, which an auditor can supply alongside the ledger. And the proposition is only as strong as the certificate scheme: if certificates can be forged, so can ledger-level verification failures. Neither proposition concerns refusal at all. A verification failure is evidence; whether it leads to refusal is decided by a separate, separately authorized act.

In the present calculus the policy judgement $\mathbf{may}$ is fixed, so the spine's non-amplification corollary holds trivially: no transition changes what any principal may do. It becomes a substantive result once delegation is added in Chapter 16, where authority can grow, and the claim is that it grows only through a delegation record.

Chapter 13

Choice, Actors, and the Arbiter

Chapter plan. The known separation results for mixed choice, and where a choice resolved by a recording arbiter falls relative to them. The choice rule of Chapter 12 leaves no record of the discarded summand; the chapter asks whether a variant that records the road not taken is more expressive, or only more informative. Contrasts the actor model's unordered arrival with total causal order.

Part IV

Linear and Session Types

Chapter 14

The Type Algebra

The untyped calculus says what happens when a step is enabled. It says nothing about programs that attempt steps that can never be enabled: a collapse of an open commitment, a second binding of the same commitment from two parallel components, a refusal by a principal without authority. Such programs simply stop, with nothing recorded. The type algebra excludes them. It does so by making the right to advance a commitment a linear resource, indexed by the commitment's identity and lifecycle state, and by stating exactly how those resources correspond to the runtime commitment map.

14.1 Types, rights, and facts

The value types are

$$A, B ::= \mathbf{1} \mid A \times B \mid A + B \mid A \rightarrow B \mid A \otimes B \mid A \multimap B \mid !A.$$

On top of them the disposition layer uses three kinds of assumption.

- *Rights* $\text{Commit}\langle c, A, s \rangle$, for a live state s : one of **Open**, **Bound**, **Transformed**, **Verified**, and **VerificationFailed**. A right is the permission to advance commitment c from state s . Rights are linear: they may be neither duplicated nor discarded.
- *Tokens* $\text{Refusal}\langle c, A, s \rangle$ and $\text{Observed}\langle c, B, k \rangle$, the results of the two terminal steps. The index s of a refusal token records the stage at which refusal occurred. Tokens are affine: they may be discarded but not duplicated.
- *Facts*: the authority policy $\text{may}(a, \kappa, x)$, failure evidence $\text{FailureEvidence}\langle c, B \rangle$ recording that verification of c returned a certified negative result, and terminal facts $\text{Terminal}\langle c, t \rangle$ with $t \in \{\text{Refused}, \text{Collapsed}\}$. Facts are persistent. A terminal fact is issued by the terminal step of c and may be consulted any number of times afterward, which is sound because terminality is monotone (lifecycle monotonicity).

The linear context holds the right to advance a commitment, never the historical fact that the commitment exists. The history and the commitment map keep c forever (Theorem 12.5); the context holds at most one right to move it.

The typing judgement for processes is

$$\Sigma; \Gamma; \Delta \vdash P,$$

where Σ is a set of names (commitment identities and obligations), Γ contains value typings and facts, and Δ contains rights and tokens. The protocol Θ offered by P is, in this edition, determined by Δ : each right $\text{Commit}\langle c, A, s \rangle$ offers the row of the refusal completion table for s , read as a protocol (Section 15.4). The general judgement $\Sigma; \Gamma; \Delta \vdash P \triangleright \Theta$ with independently specified protocols is needed only for interaction among several principals over shared channels, which this calculus does not yet have.

14.2 Operator types

Read as functions, the disposition operators have these types.

$$\begin{aligned}
&\text{bind} : \text{Commit}\langle c, A, \text{Open} \rangle \otimes A \multimap \text{Commit}\langle c, A, \text{Bound} \rangle, \\
&\text{transform} : \text{Commit}\langle c, A, \text{Bound} \rangle \otimes (A \rightarrow B) \multimap \text{Commit}\langle c, B, \text{Transformed} \rangle, \\
&\text{verify} : \text{Commit}\langle c, B, \text{Transformed} \rangle \otimes \text{Verifier}\langle c, B \rangle \\
&\quad \multimap (\exists w. \text{Commit}\langle c, B, \text{Verified}(w) \rangle \otimes \text{Evidence}\langle c, B, w \rangle) \\
&\quad \oplus (\exists \varphi. \text{Commit}\langle c, B, \text{VerificationFailed}(\varphi) \rangle \otimes \text{!FailureEvidence}\langle c, B, \varphi \rangle), \\
&\text{refuse}_s : \text{Commit}\langle c, A, s \rangle \otimes \text{Reason} \\
&\quad \multimap \text{Refusal}\langle c, A, s \rangle \otimes \text{!Terminal}\langle c, \text{Refused} \rangle, \\
&\text{collapse}_k : \text{Commit}\langle c, B, \text{Verified}(w) \rangle \\
&\quad \multimap \text{Observed}\langle c, B, k \rangle \otimes \text{!Terminal}\langle c, \text{Collapsed} \rangle.
\end{aligned}$$

Four features deserve comment.

Pop is generative, not an ordinary existential. Its external type may be written $A \rightarrow \exists c. \text{Commit}\langle c, A, \text{Open} \rangle$, but internally it is a name-generation rule analogous to restriction: its typing premise extends Σ with a name not already in it, and the continuation receives the open right for that name (rule T-POP below). An ordinary existential could be eliminated and its witness compared with other names; a generated name is fresh by construction, and that is what freshness of commitment identities rests on.

The success branch of verification carries its witness. The verified state is indexed by the witness w , and the evidence $\text{Evidence}\langle c, B, w \rangle$ is indexed by the commitment. Evidence for one commitment is never evidence for another: the index is part of the type, and under Hypothesis 2.10 it is part of the predicate too. In process typing the witness does not need a name, because collapse reads it from the runtime state; what the typing guarantees is that such a witness exists there (Lemma 14.7).

Both verification branches keep the right. The positive branch produces a verified right with its evidence; the negative branch produces a right in $\text{VerificationFailed}$ with persistent failure evidence. Neither produces a refusal token or a terminal fact, because neither is a disposition. A verifier cannot end a commitment, and the type says so.

Terminal steps issue persistent facts. Refusal and collapse consume the right and produce two things: an affine token recording the result, and a persistent fact $\text{Terminal}\langle c, t \rangle$. The fact is what compensation consults. The earlier worry, that compensation needed a certificate of terminality that no rule issued, is resolved by making the terminal rules issue it.

14.3 Context algebra

A *linear context* Δ is a finite multiset of rights and tokens. It is *well formed* when it contains at most one right for each commitment, at most one token for each commitment, and never both a right and a token for the same commitment. Write $\Delta_1 \uplus \Delta_2$ for multiset union, defined only when the result is well formed.

Lemma 14.1 (Ownership). *If $\Delta_1 \uplus \Delta_2$ is defined and Δ_1 contains a right for c , then Δ_2 contains neither a right nor a token for c .*

Proof. Otherwise the union would contain two assumptions for c of kinds that well-formedness forbids together. $\square$

For an action α with subject c , the *context step* $\Delta \xrightarrow{\alpha} \Delta'$ is defined by: a pop or compensation adds $\text{Commit}\langle c, A, \text{Open} \rangle$; binding, transformation, and successful verification replace the right for c by the right for its successor state; either kind of refusal replaces the right by the token $\text{Refusal}\langle c, A, s \rangle$; collapse replaces the right by $\text{Observed}\langle c, B, k \rangle$; a discharge leaves Δ unchanged.

Lemma 14.2 (Recombination). *If $\Delta_1 \uplus \Delta_2$ is defined, $\Delta_1 \xrightarrow{\alpha} \Delta'_1$, and either α introduces a name not mentioned in Δ_2 or Δ_1 contains the right for the subject of α , then $\Delta'_1 \uplus \Delta_2$ is defined and equals $(\Delta_1 \uplus \Delta_2)'$, the context step applied to the union.*

Proof. The context step changes only the entry for the subject of α . In the first case that entry is new to both sides. In the second, by Lemma 14.1, Δ_2 has no entry for it. Either way the changed entry lives in Δ'_1 alone and well-formedness of the union is preserved. $\square$

The unrestricted context Γ admits weakening, contraction, and exchange. The linear context admits exchange only, together with weakening by tokens.

14.4 Typing the value layer

Values are typed by $\Gamma; \Delta \vdash e : A$. The rules for the multiplicative core are

$$\frac{}{\Gamma, x:A; \emptyset \vdash x : A} \quad \frac{}{\Gamma; x:A \vdash x : A} \quad \frac{\Gamma; \Delta, x:A \vdash e : B}{\Gamma; \Delta \vdash \lambda x:A. e : A \multimap B} \quad \frac{\Gamma, x:A; \emptyset \vdash e : B}{\Gamma; \emptyset \vdash \lambda x:A. e : A \rightarrow B}$$

$$\frac{\Gamma; \Delta_1 \vdash e_1 : A \multimap B \quad \Gamma; \Delta_2 \vdash e_2 : A}{\Gamma; \Delta_1 \uplus \Delta_2 \vdash e_1 e_2 : B} \quad \frac{\Gamma; \Delta \vdash e_1 : A \rightarrow B \quad \Gamma; \emptyset \vdash e_2 : A}{\Gamma; \Delta \vdash e_1 e_2 : B}$$

with products, sums, projections, injections, and case analysis typed in the unrestricted fragment in the standard way. Transformations and verifiers used by processes are closed unrestricted terms.

Lemma 14.3 (Unrestricted substitution). *If $\Gamma, x:A; \Delta \vdash e : B$ and $\Gamma; \emptyset \vdash v : A$, then $\Gamma; \Delta \vdash e[v/x] : B$.*

Proof. Induction on the derivation of e . At an unrestricted variable x the result is the typing of v , weakened in Γ . At a rule that splits Δ , the variable x lives in Γ and so is available on both sides, and the induction hypothesis applies to each. At a binder, rename the bound variable apart from x and the free variables of v first. $\square$

Lemma 14.4 (Linear substitution). *If $\Gamma; \Delta_1, x:A \vdash e : B$ and $\Gamma; \Delta_2 \vdash v : A$, and $\Delta_1 \uplus \Delta_2$ is defined, then $\Gamma; \Delta_1 \uplus \Delta_2 \vdash e[v/x] : B$.*

Proof. Induction on the derivation of e . At a linear variable, $e = x$ and $\Delta_1 = \emptyset$, so the result is the typing of v . The linear variable x cannot occur at an unrestricted variable rule or in the body of an unrestricted abstraction, since both require an empty linear context. At linear application, $\Delta_1, x:A$ is split between the two premises and x lands in exactly one of them; apply the induction hypothesis to that premise and recombine, the resources of v joining the side that held x . At unrestricted application the argument has an empty linear context, so x is in the function premise. At linear abstraction, rename apart and apply the induction hypothesis. In no case are the resources of v duplicated, because x occurs in exactly one branch of every split. $\square$

Evaluation $e \Downarrow v$ preserves types for closed terms, by induction on the evaluation derivation using the two substitution lemmas; closed values of function type are abstractions, of product type pairs, and of sum type injections. These canonical-forms facts are standard [23] and are used without further proof.

14.5 Typing processes

The process rules are syntax directed. Write $\Gamma \vdash e : A$ for $\Gamma; \emptyset \vdash e : A$.

$$\begin{array}{c}
\text{T-NIL} \frac{\Delta \text{ contains only tokens}}{\Sigma; \Gamma; \Delta \vdash \mathbf{0}} \quad \text{T-POP} \frac{c \notin \Sigma \quad \Gamma \vdash q : A \quad \Sigma, c; \Gamma; \Delta, \text{Commit}\langle c, A, \text{Open} \rangle \vdash P}{\Sigma; \Gamma; \Delta \vdash \text{pop } q \text{ as } c.P} \\
\\
\text{T-COMP} \frac{c \notin \Sigma \quad \text{Terminal}\langle d, t \rangle \in \Gamma \quad \text{may}(a, \text{compensate}, d) \in \Gamma \quad \Gamma \vdash q : A \quad \Sigma, c; \Gamma; \Delta, \text{Commit}\langle c, A, \text{Open} \rangle \vdash P}{\Sigma; \Gamma; \Delta \vdash \text{pop}_a q \text{ as } c \text{ after } d.P} \\
\\
\text{T-BIND} \frac{\text{may}(a, \text{bind}, c) \in \Gamma \quad \Gamma \vdash b : A \quad \Sigma; \Gamma; \Delta, \text{Commit}\langle c, A, \text{Bound} \rangle \vdash P}{\Sigma; \Gamma; \Delta, \text{Commit}\langle c, A, \text{Open} \rangle \vdash \text{bind}_a c \text{ to } b.P} \\
\\
\text{T-TRANS} \frac{\Gamma \vdash f : A \rightarrow B \quad \Sigma; \Gamma; \Delta, \text{Commit}\langle c, B, \text{Transformed} \rangle \vdash P}{\Sigma; \Gamma; \Delta, \text{Commit}\langle c, A, \text{Bound} \rangle \vdash \text{transform } c \text{ with } f.P} \\
\\
\text{T-VER} \frac{\text{may}(a, \text{verify}, c) \in \Gamma \quad \Gamma \vdash g : \text{Verifier}\langle c, B \rangle \quad \Sigma; \Gamma; \Delta, \text{Commit}\langle c, B, \text{Verified} \rangle \vdash P \quad \Sigma; \Gamma, \text{FailureEvidence}\langle c, B \rangle; \Delta, \text{Commit}\langle c, B, \text{VerificationFailed} \rangle \vdash Q}{\Sigma; \Gamma; \Delta, \text{Commit}\langle c, B, \text{Transformed} \rangle \vdash \text{verify}_a c \text{ using } g \text{ then } P \text{ else } Q} \\
\\
\text{T-REF} \frac{\text{live}(s) \quad \text{may}(a, \text{refuse}, c) \in \Gamma \quad \Gamma \vdash r : \text{Reason} \quad \Sigma; \Gamma, \text{Terminal}\langle c, \text{Refused} \rangle; \Delta, \text{Refusal}\langle c, A, s \rangle \vdash P}{\Sigma; \Gamma; \Delta, \text{Commit}\langle c, A, s \rangle \vdash \text{refuse}_a c \text{ because } r.P} \\
\\
\text{T-COL} \frac{\text{may}(a, \text{collapse}, c) \in \Gamma \quad k : \text{Rule}(B) \quad \Sigma; \Gamma, \text{Terminal}\langle c, \text{Collapsed} \rangle; \Delta, \text{Observed}\langle c, B, k \rangle \vdash P}{\Sigma; \Gamma; \Delta, \text{Commit}\langle c, B, \text{Verified} \rangle \vdash \text{collapse}_{a,k} c.P} \\
\\
\text{T-AWAIT} \frac{\omega \in \Sigma \quad \text{may}(a, \text{discharge}, \omega) \in \Gamma \quad \Sigma; \Gamma; \Delta, \text{Commit}\langle c, A, s \rangle \vdash P}{\Sigma; \Gamma; \Delta, \text{Commit}\langle c, A, s \rangle \vdash \text{await}_a c \omega.P} \\
\\
\text{T-DEADLINE} \frac{\omega \in \Sigma \quad \text{may}(a, \text{discharge}, \omega), \text{may}(b, \text{clock}, \omega) \in \Gamma \quad \Sigma; \Gamma; \Delta, \text{Commit}\langle c, A, s \rangle \vdash P \quad \Sigma; \Gamma; \Delta, \text{Commit}\langle c, A, s \rangle \vdash Q}{\Sigma; \Gamma; \Delta, \text{Commit}\langle c, A, s \rangle \vdash \text{await}_a c \omega.P \text{ within}_b t \text{ else } Q} \\
\\
\text{T-PAR} \frac{\Sigma; \Gamma; \Delta_1 \vdash P \quad \Sigma; \Gamma; \Delta_2 \vdash Q}{\Sigma; \Gamma; \Delta_1 \uplus \Delta_2 \vdash P \mid Q} \quad \text{T-SUM} \frac{\Sigma; \Gamma; \Delta \vdash P \quad \Sigma; \Gamma; \Delta \vdash Q}{\Sigma; \Gamma; \Delta \vdash P + Q} \\
\\
\text{T-RES} \frac{c \notin \Sigma \quad \Sigma, c; \Gamma; \Delta \vdash P}{\Sigma; \Gamma; \Delta \vdash (\nu c)P} \quad \text{T-REP} \frac{\Sigma; \Gamma; \emptyset \vdash P}{\Sigma; \Gamma; \emptyset \vdash !P}
\end{array}$$

Here $\Gamma \vdash g : \text{Verifier}\langle c, B \rangle$ says g is a verifier whose witnesses and certificates are issued for c , and $k : \text{Rule}(B)$ says observe_k is total on B .

Several design decisions are visible in the rules. T-NIL forbids ending a process while it holds a right: a well-typed program cannot silently abandon a live commitment. T-SUM types both summands against the same context, so a choice is between alternative uses of the same right. T-REP requires an empty linear context, so no right is ever replicated. The terminal rules add the terminal fact to Γ for the continuation, and T-COMP consults it. Every attributed prefix requires its authority fact, so an unauthorized attempt cannot be written in a well-typed program at all.

14.6 Configurations and the rights–state correspondence

A typing of the process says nothing, by itself, about the runtime. The bridge between them is the following invariant. Without it the commitment map and the linear context could each be internally consistent while disagreeing with each other.

Definition 14.5 (Rights agree). $\text{RightsAgree}(A, \Delta)$ holds when, for every commitment c :

- (i) if $A(c)$ is live with state kind s , then Δ contains exactly one right for c , namely $\text{Commit}\langle c, A_c, s \rangle$, and the values held in $A(c)$ have the types that right indexes;
- (ii) if $A(c)$ is terminal or $c \notin \text{dom}(A)$, then Δ contains no right for c ;
- (iii) a token for c in Δ occurs only if $A(c)$ is terminal, and a refusal token only if $A(c)$ is refused, an observation token only if it is collapsed.

Definition 14.6 (Well-typed configuration). $\Sigma; \Gamma; \Delta \vdash \langle P, S, A, O, H \rangle$ ok when

- (C1) $\Sigma; \Gamma; \Delta \vdash P$;
- (C2) the configuration is consistent: $\text{replay}(H) = \langle A, O \rangle$;
- (C3) $\text{RightsAgree}(A, \Delta)$;
- (C4) every fact in Γ is true: the authority facts are the policy, and $\text{Terminal}\langle c, t \rangle \in \Gamma$ only if $A(c)$ is terminal of kind t ;
- (C5) $\text{dom}(A) \subseteq \Sigma$, and the names occurring free in P are in Σ .

Lemma 14.7 (Canonical forms for rights). *In a well-typed configuration, if Δ contains $\text{Commit}\langle c, B, \text{Verified} \rangle$, then $A(c) = \text{verified}(v, w)$ with v of type B and $\text{valid}(w, c, v)$. If Δ contains $\text{Commit}\langle c, B, \text{VerificationFailed} \rangle$, then $A(c) = \text{verificationFailed}(v, \varphi)$ with $\text{checkCert}(\varphi, c, v, \text{fail})$. If Δ contains $\text{Commit}\langle c, A, s \rangle$ for any live s , then $A(c)$ is in state s and $H|_c$ is a δ -path from an opening record to $A(c)$.*

Proof. By (C3) the state of c is s with correctly typed contents. By (C2) and the definition of replay, $A(c) = \delta^*(H|_c)$, which is a path; for the verified case the verification clause of δ requires $\text{valid}(w, c, v)$, for the failed case the failure clause requires the certificate check, and by stability both still hold. $\square$

Canonical forms is where the evidence discipline becomes a static guarantee: possession of a verified right is enough to know that a correctly indexed witness exists in the state, without the program ever having named it.

14.7 The proof target

The metatheory of the next chapter establishes preservation, progress, and protocol fidelity. Their synthesis is the following statement, proved in Chapter 27.

Theorem 14.8 (Typed disposition safety, target form). *Suppose $\Sigma; \Gamma; \Delta \vdash C_0 \text{ ok}$. For every finite execution $C_0 \xRightarrow{\tau} C_n$, the configuration C_n is well typed; every live commitment has exactly one right to advance it and every terminal commitment has none; every attributed transition was authorized; every witness is indexed by the commitment and value it verifies; every collapse has a complete verified ledger path; every refusal remains in the history; no commitment has two terminal dispositions; and no well-typed program contains a competing continuation for a commitment whose right it does not hold.*

Chapter 15

Preservation, Progress, and Protocol Fidelity

This chapter proves that typing is preserved by execution, that a well-typed configuration is never silently stuck, and that every execution follows the lifecycle protocol of each commitment it touches. It closes with the negative derivations: the programs the type system rejects, and why.

15.1 Structural lemmas

Lemma 15.1 (Inversion). *Each typing rule for a prefix is the only rule whose conclusion has that prefix at the root. Hence from a derivation of $\Sigma; \Gamma; \Delta \vdash \pi.P$ the premises of the corresponding rule can be read off; in particular Δ contains the right the rule consumes.*

Proof. The rules are syntax directed; only T-NIL, T-PAR, T-SUM, T-RES, and T-REP apply to non-prefix forms, and each applies to exactly one form. $\square$

Lemma 15.2 (Weakening and strengthening). *Typing is preserved by adding true facts to Γ , by adding tokens to Δ , and by adding to or removing from Σ names that occur neither free in P nor in Δ .*

Proof. Induction on the derivation. Facts are only ever consulted, tokens are absorbed by T-NIL, and names outside the free names are never inspected except by freshness premises, which remain satisfiable by choosing binders apart. $\square$

Lemma 15.3 (Equivariance of typing). *For every permutation ρ of names, $\Sigma; \Gamma; \Delta \vdash P$ implies $\rho\Sigma; \rho\Gamma; \rho\Delta \vdash \rho P$.*

Proof. Induction on the derivation; every premise that mentions names compares them only for equality or membership, which permutations preserve. $\square$

Lemma 15.4 (Congruence). *If $P \equiv Q$ then $\Sigma; \Gamma; \Delta \vdash P$ iff $\Sigma; \Gamma; \Delta \vdash Q$.*

Proof. By cases on the axioms, each in both directions. The laws for $|$ are commutativity and associativity of $\uplus$ together with T-NIL for the unit. Commutativity of $+$ exchanges the two identical-context premises of T-SUM. For $!P \equiv P | !P$, T-REP gives an empty context, and

$\emptyset \uplus \emptyset = \emptyset$. The restriction laws use Lemma 15.2 for names not free, and α -conversion uses Lemma 15.3. $\square$

15.2 Preservation

Theorem 15.5 (Preservation). *If $\Sigma; \Gamma; \Delta \vdash C \text{ ok}$ and $C \xrightarrow{\alpha} C'$, then there are $\Sigma' \supseteq \Sigma$ and $\Gamma' \supseteq \Gamma$ such that $\Sigma'; \Gamma'; \Delta' \vdash C' \text{ ok}$, where $\Delta \xrightarrow{\alpha} \Delta'$ is the context step.*

Proof. By induction on the derivation of the transition, with the process typing brought into matching shape by Lemma 15.4.

Primitive rules. By Lemma 15.1 the prefix's typing premises are available.

- *Pop.* The premise of T-POP types P under Σ, c with the open right; the operational rule chose c outside $\text{dom}(A)$, and by Lemma 15.3 we may take the binder to be that name. (C2) holds since $\delta(\perp, \text{Pop}(c, q)) = \text{open}(q)$ and $c \notin \text{dom}(A)$ means $H|_c$ was empty. (C3) holds with the new right, and $q : A$ by the premise.
- *Compensating pop.* As for pop, with the terminal fact for d in Γ matching the operational premise by (C4).
- *Bind, transform, successful verification.* Inversion gives the right for the prior state in Δ and types the continuation with the right for the next state. By (C3) the runtime state is the prior one, the operational rule moves it to the next, and δ agrees, so (C2) and (C3) hold. For transformation, $f b \Downarrow v$ with $f : A \rightarrow B$ and $b : A$ gives $v : B$ by type preservation of evaluation. For verification, v keeps its type and the witness is valid by Hypothesis 1.5.
- *Failed verification.* Inversion of T-VER types Q with the right for c in $\text{VerificationFailed}$ and the persistent fact $\text{FailureEvidence}\langle c, B \rangle$. The operational rule moves c to $\text{verificationFailed}(v, \varphi)$ and δ agrees, so (C2) and (C3) hold, and the fact is true in C' , so (C4) holds.
- *Refusal.* The right is replaced by the refusal token, the continuation is typed with the terminal fact added to Γ , and the fact is true in C' , so (C4) holds with $\Gamma' = \Gamma, \text{Terminal}\langle c, \text{Refused} \rangle$.
- *Collapse.* As for refusal, with the observation token and the collapsed fact; $o = \text{observe}_k(v)$ is defined since k is total on B .
- *Discharge and timeout.* The awaiting process holds the right for c (T-AWAIT, T-DEADLINE), so c is live by (C3), and the continuation taken, P or Q , keeps the right. The corresponding clause of δ leaves the state unchanged, so (C2) and (C3) hold with $\Delta' = \Delta$.

Contextual rules. For $P \mid Q$, T-PAR splits $\Delta = \Delta_1 \uplus \Delta_2$. The step is taken by P , and by the induction hypothesis P' is typed under Δ'_1 . The prefix that fired either introduced a name fresh for Σ , hence not mentioned in Δ_2 , or consumed a right in Δ_1 ; Lemma 14.2 gives $\Delta'_1 \uplus \Delta_2 = \Delta'$, and T-PAR retypes $P' \mid Q$ after weakening Q 's typing with the new facts and names (Lemma 15.2). For $P + Q$, the premise of T-SUM types P under the same Δ , and the induction hypothesis applies. For restriction, the induction hypothesis applies under Σ, c . For the congruence rule, Lemma 15.4.

The conditions on the other components of the configuration are carried over unchanged or updated as described in each case. $\square$

Corollary 15.6 (Rights are neither duplicated nor lost). *In every configuration reachable from a well-typed one, each live commitment has exactly one right in the linear context and each terminal commitment has none.*

Proof. (C3) is preserved by Theorem 15.5. $\square$

Corollary 15.7 (Typed path lemma). *In a well-typed configuration, if $\text{Commit}\langle c, A, s \rangle \in \Delta$ then $H|_c$ is a δ -path from an opening record to a state of kind s . It is unique in the sense that every linearization of the history's dependence order has the same projection onto c .*

Proof. The first sentence is Lemma 14.7. For the second, records of the same subject are pairwise dependent, so every linearization keeps them in the same relative order. $\square$

15.3 Progress

Progress in an ordinary calculus says a well-typed term is a value or can step. Here a third possibility is legitimate: waiting for the environment. The danger is that “waiting” becomes a place for errors to hide. It cannot here, because waiting is a positive syntactic form, the `await` prefix, which names the obligation and the principal expected to discharge it.

A process is *inert* if it is structurally congruent to a parallel composition of $\mathbf{0}$ and replications of processes with no prefix. A configuration is *waiting on* ω if every top-level prefix of its process is an `await` prefix, with or without a deadline, and one of them awaits ω . The *top-level prefixes* of P are those not beneath another prefix; a summand's prefixes count, and a replicated body's prefixes count.

Hypothesis 15.8 (Computational totality at C). *For every top-level prefix transform c with f of C with $A(c) = \text{bound}(q, b)$, evaluation of fb terminates; and for every top-level prefix $\text{verify}_a c$ using g with $A(c) = \text{transformed}(b, v)$, the verifier $g(c, v)$ returns `ok` or `fail`.*

The hypothesis is about the particular computations a configuration is about to perform, not about the lambda layer as a whole, which may be Turing complete. For the fragment without fixed points it holds automatically by strong normalization of the simply typed calculus [22].

Theorem 15.9 (Progress). *Let $\Sigma; \Gamma; \Delta \vdash C \text{ ok}$ with Hypothesis 15.8 holding at C . Then exactly one of the following holds:*

- (i) C can take a transition other than a discharge;
- (ii) C is waiting on some obligation;
- (iii) the process of C is inert, and every commitment in $\text{dom}(A)$ is terminal.

Proof. If some top-level prefix is not an `await`, we show it is enabled, so (i) holds. By Lemma 15.1 its typing premises hold, and by (C3) and (C4) the runtime agrees with them.

- A pop is always enabled: names are unbounded, so some c lies outside $\text{dom}(A)$.

- A compensating pop has $\text{Terminal}\langle d, t \rangle$ in its Γ , true by (C4), and its authority fact; a fresh name exists.
- A bind, transform, verify, refuse, or collapse prefix on c holds the right for the state it requires (inversion), and by (C3) $A(c)$ is in that state. Authority is a premise of the typing. A transform or verify terminates by Hypothesis 15.8. A collapse finds a valid witness in the state by Lemma 14.7 and a defined observation since k is total on the value's type.
- A prefix under a sum, restriction, or replication fires through the corresponding contextual rule; for replication, through $!R \equiv R \mid !R$.

If every top-level prefix is an **await** and there is at least one, (ii) holds. If there is none, the process is inert; its typing derivation ends in instances of T-NIL, so Δ contains only tokens, and by (C3) no commitment is live.

The three cases are exclusive by definition of the second and third, and since a configuration with an enabled non-await prefix is neither waiting nor inert. $\square$

The third case is the formal statement that a well-typed program cannot silently abandon a commitment: if nothing can happen and nothing is being waited for, every commitment has been disposed of. A process attempting to collapse an unverified commitment is not waiting and not inert; it is untypable.

15.4 Protocol fidelity

Read the refusal completion table as a protocol: for each live state s , the protocol offers the actions in $\text{Next}(s)$, and each action leads to the protocol of its successor state. For a single commitment this is the session type

$$\begin{aligned} \mathcal{S} = & \text{open}; (\text{refuse} \oplus \text{bind}; (\text{refuse} \oplus \text{transform}; (\text{refuse} \\ & \oplus \text{verify}^+; (\text{refuse} \oplus \text{collapse}) \oplus \text{verify}^-; \text{refuse}))), \end{aligned}$$

where verify^+ and verify^- are the two outcomes of verification, and after a negative outcome the only continuation is a refusal.

Theorem 15.10 (Protocol fidelity). *Along every execution from a well-typed configuration, the sequence of actions whose subject is c is a trace of $\mathcal{S}$.*

Proof. By Theorem 15.5, each step updates the linear context by the context step, which follows exactly one edge of the table for the subject of the action. The first action on c is its pop or compensating pop. $\square$

This is fidelity for the protocol of a single commitment. Protocols that coordinate several principals over channels, with a protocol and its dual held by different parties, need a calculus with communication; the correspondence with the session-type literature, including Wadler's propositions-as-sessions reading, is taken up when that extension is made.

15.5 Negative derivations

The type system's value is measured by what it rejects.

Proposition 15.11 (Rejected programs). *There is no typing derivation for any of the following, in any context in which the named commitments' rights are as described.*

- (a) *A collapse of a commitment whose right is open, bound, or transformed.*
- (b) *A bind of a commitment that has been refused: only a token and a fact remain, and T-BIND requires a right.*
- (c) *Two uses of one right: either in parallel, since $\Delta_1 \uplus \Delta_2$ cannot contain two rights for c , or in sequence, since after the first use the right has moved to a different state.*
- (d) *Verifying c_1 with a verifier indexed by $c_2 \neq c_1$: T-VER requires $\text{Verifier}\langle c_1, B \rangle$.*
- (e) *Any attributed prefix without the corresponding authority fact.*
- (f) *A process that ends holding a live right: T-NIL admits only tokens.*

Proof. Each by Lemma 15.1: the only rule for the prefix in question has a premise the described context cannot satisfy. $\square$

Case (c) is the static half of terminal exclusivity. Operationally, two components racing to refuse and collapse the same commitment produce exactly one terminal record (Chapter 12); statically, a well-typed program cannot contain the race, because only one component can hold the right.

Chapter 16

Authority as a Modality

The core calculus treats authority as a fixed policy: a set of facts $\text{may}(a, \kappa, x)$ that no rule changes. That is enough for the safety theorems, and it makes non-amplification trivial. Real systems transfer authority, and a transfer is exactly the kind of event that must be recorded, authenticated, and replayable. This chapter adds delegation as a transfer of an exclusive capability, proves that it conserves authority rather than copying it, and then applies the resulting distinctions to a contemporary case.

16.1 Exclusive authority as a linear capability

Two kinds of authority are distinguished. *Shared* authority is a policy fact $\text{may}(a, \kappa, x)$ in Γ , as before; any number of principals may hold it and nothing transfers it. *Exclusive* authority is a linear capability

$$\text{Cap}\langle a, c, \kappa \rangle$$

held in Δ : the right of principal a , and of no one else, to perform acts of kind κ on commitment c . At runtime the configuration carries an ownership map Own from pairs (c, κ) to the single principal who currently holds the exclusive capability. An attributed rule whose authority premise was $\text{may}(a, \kappa, c)$ now accepts either the shared fact or current ownership, $\text{Own}(c, \kappa) = a$. Exercising a capability does not consume it; the typing rules pass it on to the continuation unchanged, so linearity forbids duplication and discarding without forbidding use.

Exclusive capabilities enter the world only through a grant record $\text{Grant}(g, a, c, \kappa, \sigma)$ issued by a root authority g , and move only through delegation.

16.2 Delegation

The prefix $\text{delegate}_{a \rightarrow b}\langle c, \kappa \rangle.P$ transfers a 's exclusive capability over (c, κ) to b . Its typing rule consumes the delegator's capability and gives the continuation the delegate's:

$$\frac{\Sigma; \Gamma; \Delta, \text{Cap}\langle b, c, \kappa \rangle \vdash P}{\Sigma; \Gamma; \Delta, \text{Cap}\langle a, c, \kappa \rangle \vdash \text{delegate}_{a \rightarrow b}\langle c, \kappa \rangle.P}.$$

Its operational rule checks current ownership and an authentication by the delegator, moves ownership, and appends an authenticated record:

$$\frac{\text{Own}(c, \kappa) = a \quad \text{checkSig}(\sigma, a, \langle b, c, \kappa \rangle)}{\langle \text{delegate}_{a \rightarrow b} \langle c, \kappa \rangle . P, \dots, \text{Own}, H \rangle \xrightarrow{\text{delegate}(a, b, c, \kappa, \sigma)} \langle P, \dots, \text{Own}[(c, \kappa) \mapsto b], H \cdot \text{Delegate}(a, b, c, \kappa, \sigma) \rangle}.$$

Replay reconstructs **Own** from grant and delegation records and accepts **Delegate**(a, b, c, κ, σ) only if, at that point in the ledger, a owns (c, κ) , the signature authenticates the record, and the commitment and kind match the signed payload. Because ownership moves, a delegation cannot be replayed a second time by its former owner: after it, a no longer owns what it would be delegating. Scope and expiry constraints, where a policy uses them, are further conjuncts of the same check.

16.3 Results

Write $\text{Owners}_H(c, \kappa)$ for the set of principals holding the exclusive capability for (c, κ) after replay of H .

Proposition 16.1 (Delegation authenticity). *If replay of H is defined and H contains $\text{Delegate}(a, b, c, \kappa, \sigma)$, then at that point in H the delegator a owned (c, κ) and σ authenticates a 's transfer to b .*

Proof. These are the conditions under which replay accepts the record. $\square$

Proposition 16.2 (Conservation). *For every (c, κ) that has been granted, $|\text{Owners}_H(c, \kappa)| = 1$ after every ledger prefix, and a delegation from a to b changes it by*

$$\text{Owners}_{H'}(c, \kappa) = (\text{Owners}_H(c, \kappa) \setminus \{a\}) \cup \{b\}.$$

In particular delegation never increases the number of holders of an exclusive capability.

Proof. A grant sets the owner of an ungranted pair. A delegation is accepted only from the current owner and replaces that owner by the delegate. No other record changes **Own**. $\square$

Proposition 16.3 (Preservation under transfer). *Extend well-typed configurations with the condition $\text{CapsAgree}(\text{Own}, \Delta)$: $\text{Cap}\langle a, c, \kappa \rangle \in \Delta$ exactly when $\text{Own}(c, \kappa) = a$. Then Theorem 15.5 holds for the calculus with delegation.*

Proof. The new case is delegation. Inversion of its typing rule gives the delegator's capability in Δ and types the continuation with the delegate's; the operational rule moves ownership from a to b , so **CapsAgree** is maintained, and the rights, commitment map, and history conditions are untouched except for the appended record, which replay accepts. Every other case is as before, with the authority premise now discharged either by a shared fact or, by **CapsAgree**, by a capability in the context. Context splitting keeps each capability in exactly one component, by the same ownership argument as for rights (Lemma 14.1). $\square$

Proposition 16.4 (Chain soundness). *If an attributed record in H was authorized by an exclusive capability, there is a chain*

$$\text{Grant}(g, a_0, c, \kappa, \sigma_0) ; \text{Delegate}(a_0, a_1, c, \kappa, \sigma_1) ; \dots ; \text{Delegate}(a_{n-1}, a_n, c, \kappa, \sigma_n)$$

of authenticated records earlier in H , with a_n the principal of the record.

Proof. By induction on the ledger, ownership of (c, κ) at any point is the end of the chain of records that set it, each accepted only from the previous owner (Propositions 16.1 and 16.2). $\square$

Together these give non-amplification in the presence of delegation: after any execution, the principals holding authority of kind κ over c are those holding the shared fact, which no rule adds, together with the single current owner of the exclusive capability, whose authority is traced to a grant by an authenticated chain.

Confinement. The results depend on capabilities having no other source. If the lambda layer could produce a value of capability type, for instance a function returning $\text{Cap}(b, c, \kappa)$ from nothing, conservation would fail at once: a transformation could mint authority. The calculus therefore gives capabilities no expression form. They appear only in linear contexts, introduced by grants and moved by delegation, and no value typing rule mentions them. That is a design constraint the propositions rely on, and any extension that lets computation handle capabilities as data must re-establish it.

What is left out. Revocation is not in the core. A revocation can race with the exercise of the capability it revokes, and deciding which happened first requires a declared linearization point, the same obligation that operational terminal exclusivity places on the sequencer (Hypothesis 12.4). It is a later extension.

16.4 Case study: evidence production is not disposition authority

A recent study of proposal selection at a large neutron-scattering facility gives a clean contemporary case for the distinctions this chapter formalizes [20]. The authors used a language model to compare every pair of proposals in a review cycle, aggregated the pairwise preferences into a ranking with a Bradley–Terry model, and compared the result with the historical human rankings. Across cycles the rank correlations ranged from roughly 0.2 to 0.8, typically around 0.6, and proposals on which the two rankings diverged sharply were flagged for the review committee. They also used embedding similarity between proposals as a cheap triage signal for possible duplicates, and they are explicit that this signal is not a validated redundancy classifier: a high-similarity pair may be a resubmission, related work, or merely shared vocabulary, and telling these apart needs expert judgement. Their stated position is that the model should augment the committee rather than replace it, and that the committee keeps final authority over flagged divergences.

Two features of the evaluation bear on authority as well as on accuracy. Publication outcomes, used to compare the rankings’ effectiveness, exist only for accepted proposals, and acceptance was historically decided by the human ranking; the authors note that this biases the comparison toward the human ranking. And where either ranking had ties, the other method’s score was used as a secondary sorting key, so the two rankings are not wholly independent in tied cases. Neither point undoes the feasibility result. Both illustrate that evidence about an evidence producer’s reliability can itself be shaped by earlier dispositions, which is exactly the kind of dependence a ledger makes auditable.

The study involves four roles, and the calculus can keep them apart:

- a *signal producer*, such as the embedding-similarity score, which flags candidates for attention and has no evidential standing;
- an *evidence producer*, such as the model’s pairwise comparisons aggregated by Bradley–Terry, which produces a ranking with provenance that others may consult;
- a *verifier*, which checks a claim against an independent truth condition;
- a *disposition authority*, which accepts or refuses.

In the study the model is an evidence producer. Pairwise preference is not verification of scientific merit against an independent condition, and nothing in the study treats it as such. A committee may choose to verify the ranking *procedure*, for instance by checking position-swap consistency, but that is a verification of the procedure, performed under the committee’s authority, not the model verifying the proposals.

Each role corresponds to a class of records it is able to write:

Role	Authority kind	Records it can write
signal producer	(annotation)	annotation-channel entries only; nothing causal
evidence producer	discharge	Discharged records; lifecycle-neutral
verifier	verify	Verify and VerificationFailed records; never terminal
disposition authority	collapse, refuse	Collapse and explicit Refuse records

Proposition 16.5 (Non-derivability of authority). *In the calculus with a static policy, no transition and no typing rule adds an authority fact, and every attributed record requires the authority kind $\text{auth}(\alpha)$ of Chapter 12. Hence a principal whose facts are exactly those of one role writes, in every execution, only records in that role’s row of the table. In particular*

$$\text{signal} \not\Rightarrow \text{evidence}, \quad \text{evidence} \not\Rightarrow \text{verify}, \quad \text{verify} \not\Rightarrow \text{refuse}, \quad \text{verify} \not\Rightarrow \text{collapse}.$$

Proof. The policy is a fixed set of facts consulted only by membership, and no rule’s conclusion adds to it. Theorem 12.9 gives the required kind for each attributed record, and the table lists, for each kind, the records whose required kind it is. $\square$

These are non-derivability results from the rules, not empirical claims that a model can never hold a later role. A policy may explicitly grant verification authority to a model. What the proposition rules out is authority arising merely because a component produced a score or a ranking. With delegation the same holds in stronger form: a principal can come to hold exclusive authority only at the end of an authenticated chain of grants and delegations (Proposition 16.4), each of which is an explicit, recorded act by the previous holder.

The fourth relation depends on a design decision taken for exactly this reason. In an earlier version of the calculus a failed verification refused its commitment automatically, which gave every verifier a refusal power through the negative branch even though the type system denied it explicit refusal authority. Verification now leads to `verificationFailed`, a live state, and only a principal holding refusal authority can end the commitment from there. Verification classifies evidence; disposition determines continuation. The cost is one more live state, and one more way for a commitment to wait: a failed commitment whose disposition authority never acts stays live, and Chapter 24 lists that as a counterexecution rather than hiding it.

The encoding. Give the model only discharge authority. Each proposal awaits an obligation ω_{rank} , the model’s ranking, which the model discharges with an authenticated certificate; the committee holds verification, collapse, and refusal:

pop q as c . bind _{s} c to *proposal*. await _{m} c ω_{rank} . transform c with *summary*.
 verify _{h} c using *review* then collapse _{h, alloc} c . **0** else **0**,

with policy $\text{may}(s, \text{bind}, c)$ for the intake principal s , $\text{may}(m, \text{discharge}, \omega_{\text{rank}})$, and $\text{may}(h, \kappa, c)$ for $\kappa \in \{\text{verify}, \text{collapse}, \text{refuse}\}$. A flagged divergence becomes a second obligation on which the committee’s own verification waits.

Corollary 16.6 (Signals and evidence do not dispose). *In every execution of this encoding, every record attributed to m is a discharge record, and no record attributed to m changes any commitment’s lifecycle state.*

Proof. Proposition 16.5 for the evidence-producer row, with the discharge clause of δ returning the incoming state. $\square$

Provenance of the ranking. A rank is a number, and a number looks more determinate than the process that produced it. The discharge certificate for ω_{rank} should therefore name, in the claim it hashes, everything the rank depends on: the comparison set, the model and prompt versions, the ordering controls used against positional bias, the aggregation parameters, and whether a human score was used as a tie-breaker. The last item matters for this study in particular, since its tie-breaking rule made the two rankings partly dependent. With those fields in the certificate, a later reader of the ledger can tell which ranking the committee saw and how it was made; without them, the ledger records that a ranking was delivered but not what it was evidence of.

In a typed program the separation is static as well: any prefix in which m binds, verifies, collapses, or refuses fails to type, since each rule requires the corresponding authority fact. The committee’s action supplies the attributable disposition. If the model’s ranking were treated as the decision itself, synthetic evidence would have become synthetic authority, and the ledger would show exactly where: a terminal record attributed to a principal that should not have held the authority to write it. The general lesson is that when a system grants an automated component authority, the question is not what the component is meant to do but which records it is able to write.

Part V

The Disposition Calculus

Chapter 17

The Ledger Path Lemma

The theorems of this Part all concern what a history must contain once the commitment map says something about a commitment. They share one proof. Rather than repeat the same induction five times, this chapter proves a single invariant relating the ledger to the map, and the next three chapters read the individual results off it.

The projection $H|_c$, the subject of a record, and the lifecycle step function δ were defined in Section 1.4. The whole chapter turns on one feature of δ worth restating: it is a partial fold, not a filter. Every clause either advances the lifecycle or, for a discharge, checks authenticated evidence against a live state; none passes a record through unexamined. So if $\delta^*(H|_c)$ is defined, every record of $H|_c$ was consumed by exactly one step of the fold. A history cannot fold correctly while carrying a record the fold ignored.

17.1 The lemma

Call an initial configuration $C_0 = \langle P_0, S_0, A_0, O_0, H_0 \rangle$ *ledger consistent* when $\delta^*(H_0|_c) = A_0(c)$ for every $c \in \text{dom}(A_0)$ and $H_0|_c$ is empty for every $c \notin \text{dom}(A_0)$. The empty configuration, with A_0 , O_0 , and H_0 all empty, is ledger consistent.

Lemma 17.1 (Ledger path lemma). *If C_0 is ledger consistent and $C_0 \rightarrow^* C = \langle P, S, A, O, H \rangle$, then for every commitment identity c :*

- (i) $c \in \text{dom}(A)$ if and only if $H|_c$ is nonempty;
- (ii) if $c \in \text{dom}(A)$ then $\delta^*(H|_c)$ is defined and equals $A(c)$.

Proof. By induction on the length of the execution. The base case is ledger consistency. For the step $C \xrightarrow{\alpha} C'$, history monotonicity gives $H' = H \cdot \rho$ with $\rho = \text{record}(\alpha)$, and let $s = \text{subj}(\rho)$.

For $c \neq s$, we have $H'|_c = H|_c$ because ρ is not in the projection, and $A'(c) = A(c)$ with $c \in \text{dom}(A') \iff c \in \text{dom}(A)$ by Theorem 12.5. Both clauses carry over.

For $c = s$, we have $H'|_s = H|_s \cdot \rho$ and must show $\delta(\delta^*(H|_s), \rho) = A'(s)$, where the inner fold is taken to be $\perp$ when $H|_s$ is empty. We examine the primitive rule at the root of the derivation.

- *Pop or compensating pop.* The premise $s \notin \text{dom}(A)$ and clause (i) give $H|_s$ empty, so the fold is $\perp$, and δ returns $\text{open}(q) = A'(s)$. Clause (i) now holds since $H'|_s = \langle \rho \rangle$.

- *Bind.* The premise $A(s) = \text{open}(q)$ and clause (ii) give $\delta^*(H|_s) = \text{open}(q)$, and δ applied to $\text{Bind}(a, s, b)$ returns $\text{bound}(q, b) = A'(s)$.
- *Transform.* From $A(s) = \text{bound}(q, b)$ the fold yields that state, the record carries the same b because the rule reads it from $A(s)$, and δ returns $\text{transformed}(b, v)$.
- *Verification, success.* From $A(s) = \text{transformed}(b, v)$, the record carries that v , the premise $\text{valid}(w, s, v)$ satisfies the side condition, and δ returns $\text{verified}(v, w)$.
- *Verification, failure, and explicit refusal.* Both premises place s in a live state, satisfying the side condition, and δ returns $\text{refused}(r)$ with the recorded r .
- *Discharge and timeout.* The premises give a live state and an authentic certificate, which are the side conditions of the corresponding clause, and δ returns the same state, as the rule leaves $A(s)$ unchanged.
- *Collapse.* From $A(s) = \text{verified}(v, w)$, the record carries that w , the premise $o = \text{observe}_k(v)$ satisfies the side condition, and δ returns $\text{collapsed}(o)$.

In each case the rule's premises are precisely the conditions under which δ is defined on the new record, and its conclusion is precisely δ 's value. $\square$

The proof has a shape worth noticing. Nothing in it is specific to refusal, binding, or collapse; it only checks that each operational rule and the corresponding clause of δ say the same thing. That is the sense in which the rest of this Part is “nearly forced by the preceding architecture”. The substance was spent in designing the rules so that each premise is a check on the ledger and each conclusion is a record.

17.2 Causal order

The ancestry theorems speak of one record causally preceding another, $e \prec_H e'$, and the spine asks that ledger order count only “where order represents a true dependency”. The path lemma supplies the dependency for records of the same commitment: each record in $H|_c$ could be appended only because the previous one had produced the state its rule required. For records of different commitments, Chapter 21 shows that ledger order is generally not a dependency at all, since adjacent records of distinct subjects may be exchanged without changing anything replay can recover. We therefore take $\prec_H$ to be the dependence order of Definition 21.9, and note that every ancestry chain proved in this Part lies within a single projection $H|_c$, where ledger order and $\prec_H$ agree.

Chapter 18

Refusal, Binding, and Evidence

Throughout this chapter and the next, C_0 is ledger consistent, $C_0 \rightarrow^* C = \langle P, S, A, O, H \rangle$, and Hypothesis 1.4 holds. Write $\text{Open}(c, q)$ for either opening record, $\text{Pop}(c, q)$ or $\text{Compensates}(a, c, q, d)$.

18.1 Refusal preservation

Refusal must be treated as a successful terminal disposition, not as a missing execution.

Theorem 18.1 (Refusal preservation). *If $A(c) = \text{refused}(r)$ at some configuration C_i of an execution, then at every later configuration C_j ,*

$$A_j(c) = \text{refused}(r) \quad \text{and} \quad \text{Refuse}(a, c, r) \in H_j$$

for the principal a that refused it.

Proof. The first equation is lifecycle monotonicity, since $\text{refused}(r)$ is $\preceq_D$ -maximal and is related only to itself, together with the observation that no rule rewrites the reason of a terminal state. For the second, the path lemma at C_i gives $\delta^*(H_i|_c) = \text{refused}(r)$; the only clause of δ producing that value is the refusal clause, so the last record of $H_i|_c$ is $\text{Refuse}(a, c, r)$ for some a ; and history monotonicity carries it into every H_j . $\square$

The spine stated refusal preservation with an exception for a compensating successor. In this calculus the exception is not needed: compensation introduces a new identity and leaves the predecessor's state untouched (Theorem 12.5), so the refused commitment remains refused forever and it is the *successor* that carries the matter forward. A later acceptance cannot erase the refusal because there is no rule that could even address it.

18.2 Binding integrity

A binding should establish a durable relation without confusing association with identity.

Corollary 18.2 (Binding integrity). *If $A(c) = \text{bound}(q, b)$, then*

$$H|_c = \langle \text{Open}(c, q), \text{Bind}(a, c, b) \rangle$$

for some principal a . In particular the binding record is unique, it is preceded by the opening record of the same commitment, and it binds the value b held in the state.

Proof. By the path lemma, $\delta^*(H|_c) = \text{bound}(q, b)$. Inspecting δ , the only record sequences folding from $\perp$ to a bound state are an opening record followed by one binding record. $\square$

Commitment identities and expression values are disjoint sorts (Chapter 5), so c and b cannot become interchangeable because their concrete encodings coincide. The stronger clause of the spine, that an exclusive capability is bound to at most one live commitment, is not provable in the untyped calculus, where nothing marks a value as exclusive. It is a consequence of linearity and is proved in Part IV.

18.3 Witness ancestry

Corollary 18.3 (Witness ancestry). *If $A(c) = \text{verified}(v, w)$, then*

$$H|_c = \langle \text{Open}(c, q), \text{Bind}(a, c, b), \text{Transform}(c, f, b, v), \text{Verify}(a', c, v, w) \rangle$$

for some q, a, b, f, a' , and $\text{valid}(w, c, v)$ holds.

Proof. The only record sequences folding from $\perp$ to $\text{verified}(v, w)$ are an opening, a binding, a transformation, and a verification, with the matching conditions of δ forcing the b of the binding to equal the input of the transformation and the output v of the transformation to equal the value verified. The side condition of the verification clause is $\text{valid}(w, c, v)$, and by stability it still holds. $\square$

The four records form a chain in $\prec_H$ because they lie in one projection. The spine's requirement that a witness “witnesses” the transformation is realized by indexing: w is valid for the pair (c, v) , and v is the output recorded in $\text{Transform}(c, f, b, v)$. A witness produced for another commitment, or for another value of this one, fails the matching conditions of δ and has no rule by which it could enter this chain. What indexing does *not* rule out is a verifier g that issues valid witnesses too liberally; the calculus guarantees provenance of evidence, not the quality of the predicate, and that limit is stated here so that no later chapter is read as claiming otherwise.

Chapter 19

Collapse Soundness and Terminal Exclusivity

This is the principal local safety theorem of the untyped calculus.

Theorem 19.1 (Collapse soundness). *If $A(c) = \text{collapsed}(o)$, then*

$$H|_c = \langle \text{Open}(c, q), \text{Bind}(a, c, b), \text{Transform}(c, f, b, v), \text{Verify}(a', c, v, w), \text{Collapse}(a'', c, k, w, o) \rangle,$$

with $\text{valid}(w, c, v)$ and $o = \text{observe}_k(v)$, and $(c, k, o) \in O$.

Proof. By the path lemma and inspection of δ , the only sequences folding to $\text{collapsed}(o)$ are a sequence folding to some $\text{verified}(v, w)$ followed by a collapse record carrying the same w and satisfying $o = \text{observe}_k(v)$. Corollary 18.3 applied to that prefix supplies the first four records and validity. The observation (c, k, o) was added to O by the collapse rule that appended the last record, and no rule removes observations. $\square$

The theorem rules out each of the failures the spine names. There is no collapse *ex nihilo*, because the fold requires an opening record. There is no collapse from an unrelated witness, because δ requires the collapse record's witness to be the one held in the verified state, which Corollary 18.3 ties to this commitment and value. There is no collapse after refusal, because δ is undefined on any record following a refusal. And there is no collapse whose ancestry has been omitted, because the projection is a subsequence of an append-only history and the fold must pass through every stage.

Corollary 19.2 (Terminal exclusivity, history form). *For every commitment c , the projection $H|_c$ contains at most one terminal record, and if it contains one, that record is last. In particular*

$$\text{Refuse}(a, c, r) \in H \implies \text{Collapse}(a', c, k, w, o) \notin H.$$

Proof. By the path lemma $\delta^*(H|_c)$ is defined. Since δ is undefined on every record whose incoming state is terminal, no record follows a terminal record in a sequence on which δ^* is defined. $\square$

Reconsideration therefore never produces two terminal records for one identity. It produces a chain of distinct identities, which is the subject of the next chapter.

Remark 19.3 (Which collapse rule). The spine stated collapse soundness with a single condition $\text{denotes}(v, o)$. Here the condition is $o = \text{observe}_k(v)$ for the declared rule k , recorded in the ledger. This is where the disposition calculus meets the kernel's rule-indexed collapse, and it bears directly on the third comparative claim: two commitments with equal verified values may collapse under different rules to different observations, and the ledger says which rule produced which. Whether this is more than a relabelling of a family of observational equivalences is the question of Chapter 25.

Chapter 20

Compensation

Compensation must add history rather than revise it. It is the formal boundary between historical reversibility, which the calculus forbids, and material repair, which it permits.

Theorem 20.1 (Compensating continuation). *If $C \xrightarrow{\text{compensate}(a, c', q, c)} C'$, then*

- (i) $c' \neq c$;
- (ii) $H'|_c = H|_c$ and $A'(c) = A(c)$, so the terminal record of c is present in H' and unchanged;
- (iii) $\text{Compensates}(a, c', q, c) \in H'$ and $H \sqsubseteq H'$;
- (iv) $\text{may}(a, \text{compensate}, c)$ held in C .

Proof. The rule's premises give $c' \notin \text{dom}(A)$ and $\text{terminal}(A(c))$, so $c \in \text{dom}(A)$ and (i) follows. Part (ii) is Theorem 12.5 together with the fact that the new record's subject is c' ; by Corollary 19.2 and the path lemma the last record of $H|_c$ is terminal. Part (iii) is history monotonicity and (iv) is Theorem 12.9. $\square$

The spine required that “no projection may entail that the original disposition never occurred”. That clause can be given a precise form.

Definition 20.2 (Supersession). *Write $c \rightsquigarrow c'$ when $\text{Compensates}(a, c', q, c) \in H$. The current successor of c is the last element of the longest chain $c \rightsquigarrow c_1 \rightsquigarrow \dots \rightsquigarrow c_n$ in H , if such chains are unique, and the set of chain endpoints otherwise.*

Proposition 20.3 (Supersession is a view, not a revision). *The relation $\rightsquigarrow$ is acyclic. For every commitment c and every extension $H \sqsubseteq H'$ produced by execution, $\delta^*(H'|_c) = \delta^*(H|_c)$ whenever c was terminal in H .*

Proof. If $c \rightsquigarrow c'$ then the opening record of c' appears later in H than the opening record of c , since c was already in the domain when c' was introduced. Opening positions strictly increase along any chain, so no chain returns to its start. For the second statement, a terminal c admits no further record in its projection by Corollary 19.2, so $H'|_c = H|_c$. $\square$

Ancestry that runs through compensation crosses projections: the record

$$\text{Compensates}(a, c', q, c)$$

lies in $H|_{c'}$ while referring to c . The path lemma covers each projection separately, so the crossing needs its own statement. Write $c \triangleleft_H c'$ when $\text{Compensates}(a, c', q, c) \in H$.

Lemma 20.4 (Compensation ancestry). *Let H be the history of an execution from a consistent configuration. If $c \triangleleft_H c'$, then $H|_c$ ends in a terminal record, and that record precedes the compensation record both in H and in the dependence order $\prec_H$. Consequently the ancestry of any commitment is a finite alternating sequence*

$$\pi_{c_0} \triangleleft \pi_{c_1} \triangleleft \cdots \triangleleft \pi_{c_n},$$

where each π_{c_i} is the lifecycle path $H|_{c_i}$, each π_{c_i} with $i < n$ ends in a terminal record, and every record is $\prec_H$ -after every record to its left.

Proof. At the configuration where the compensation record was appended, the rule's premise made $A(c)$ terminal, so by the path lemma the last record of $H|_c$ at that point was terminal, and it was already in the history. The compensation record refers to c , so it is dependent on every record of $H|_c$ (Definition 21.9), and hence $\prec_H$ -after them. Records within one projection are $\prec_H$ -ordered because they share a subject. Finiteness and the absence of repeats follow from acyclicity of $\triangleleft_H$ (Proposition 20.3). $\square$

A projection that reports only current successors is legitimate; it is a choice of what to display. What no execution can produce is a history from which the predecessor's disposition has become unrecoverable, because that disposition is a function of $H|_c$ and $H|_c$ never changes again. The distinction between the two is exactly the distinction between a **Collapse** rule and an edit.

Chains need not be unique. Two principals, each authorized, may compensate the same refused commitment, producing two successors. The calculus records both and does not choose; whether a policy should forbid this, or a later collapse should arbitrate between successors, is listed in Appendix E.

Part VI

Replay, Progress, and Concurrency

Chapter 21

Replay

21.1 Replay as a fold

Replay reconstructs the disposition and observation state from the ledger. Its state is a pair $X = \langle A, O \rangle$ and it applies one record at a time:

$$\text{apply}(\langle A, O \rangle, \rho) = \begin{cases} \langle A[s \mapsto \delta(A(s), \rho)], O \cup \{(s, k, o)\} \rangle & \text{if } \rho = \text{Collapse}(a, s, k, w, o), \\ \langle A[s \mapsto \delta(A(s), \rho)], O \rangle & \text{otherwise,} \end{cases}$$

where $s = \text{subj}(\rho)$ and $A(s)$ is read as $\perp$ when $s \notin \text{dom}(A)$. For a compensation record $\text{Compensates}(a, s, q, d)$, **apply** is additionally defined only when $A(d)$ is terminal, matching the operational premise. **apply** is undefined whenever δ is, so replay of a ledger that no execution could have produced fails rather than silently repairing it. Then

$$\text{replay}_{X_0}(H) = \text{the left fold of } \text{apply} \text{ over } H \text{ from } X_0.$$

Theorem 21.1 (Replay correspondence). *Let C_0 be ledger consistent with $\text{replay}_{\langle \emptyset, \emptyset \rangle}(H_0) = \langle A_0, O_0 \rangle$. If $C_0 \rightarrow^* \langle P, S, A, O, H \rangle$, then*

$$\text{replay}_{\langle \emptyset, \emptyset \rangle}(H) = \langle A, O \rangle.$$

Proof. By induction on the execution. For a step appending ρ with subject s , **apply** updates the map at s by δ , which by the path lemma agrees with the operational update, and leaves every other key unchanged, which agrees with Theorem 12.5. For a compensation record the additional condition on $A(d)$ is the operational premise. The observation component changes only on a collapse record, by the same triple the collapse rule adds to O . $\square$

The store S is absent from the statement, and not by oversight. In the rules of Chapter 12 no transition changes S ; the component is carried for the extensions in which transformations read and write mutable resources, receive external input, or behave nondeterministically. In those extensions S is reconstructible exactly when every state-affecting input is itself recorded, and not otherwise. The present theorem claims only the part that holds unconditionally.

Replay also serves as a checker. Because **apply** is partial, $\text{replay}(H)$ is defined only on histories whose every projection is a legal lifecycle path. By the path lemma every history produced by execution passes this check, so a ledger received from elsewhere can be audited by replaying it: success certifies that each commitment's records form an admissible path, with valid witnesses, matching data, and at most one terminal record, provided Hypothesis 1.4 holds at the auditor.

21.2 Consistency as an invariant

The replay theorem is stated for executions from a ledger-consistent start. It is cleaner to state its content as an invariant on single configurations, since that is what a running or restored system needs.

Definition 21.2 (Consistency). *A configuration $C = \langle P, S, A, O, H \rangle$ is consistent, $\text{Consistent}(C)$, when $\text{replay}(H) = \langle A, O \rangle$.*

Proposition 21.3 (Consistency is established and preserved). *The empty configuration is consistent. If $\text{Consistent}(C)$ and $C \xrightarrow{\alpha} C'$, then $\text{Consistent}(C')$.*

Proof. The first statement is immediate. For the second, let ρ be the appended record with subject s . By consistency $A(s)$ is the replayed state of s , the rule's premises are exactly the conditions under which $\delta(A(s), \rho)$ is defined, and its conclusion writes that value. The map is unchanged elsewhere (Theorem 12.5), and the observation set changes only on collapse, by the triple replay adds. $\square$

Theorem 21.1 is the iterate of this proposition. A system that starts empty is consistent forever. A system resumed from stored state is admitted only through a checked initialization, which combines the consistency test with the completeness anchor of the next section:

$$\frac{\text{replay}(H) = \langle A, O \rangle \quad \text{ValidAnchor}(H, \chi)}{\text{init}(A, O, H, \chi) \text{ ok}}.$$

One invariant then covers empty systems, restored systems, and embedded long-running systems alike, and the ledger path lemma applies to all of them.

21.3 Completeness and anchors

Replay as a checker certifies that each commitment's records form an admissible path. It cannot certify that the ledger is complete.

Theorem 21.4 (Prefix indistinguishability). *If H is the history of an execution from a ledger-consistent configuration, then every prefix $H' \sqsubseteq H$ is itself the history of an execution from the same configuration, and $\text{replay}(H')$ is defined. Consequently no predicate on histories alone separates complete ledgers from truncated ones.*

Proof. Stop the execution after its first $|H'|$ transitions. By history monotonicity its history at that point is H' , and by Theorem 21.1 its replay is defined. A predicate that accepted H as complete and rejected H' as truncated would reject the complete history of this shorter execution. $\square$

Truncation is only the simplest case. The laboratory found interior deletions that replay cannot detect either (Chapter 2), and the general statement is this.

Theorem 21.5 (Subhistory indistinguishability). *There are histories H produced by execution and proper subsequences H' of H , not prefixes of it, such that $\text{replay}(H)$ and $\text{replay}(H')$ are both defined and H' is itself the history of an execution. Hence replay validity certifies the local admissibility of the records presented, not their completeness relative to what actually occurred.*

Proof. Take $H = \text{Pop}(c, q); \text{Bind}(a, c, b); \text{Transform}(c, f, b, v); \text{Refuse}(a, c, r)$ and delete the transformation. The remainder $\text{Pop}(c, q); \text{Bind}(a, c, b); \text{Refuse}(a, c, r)$ is the history of the execution that refuses from the bound state, which the explicit refusal rule permits. Both replay. $\square$

Detection therefore needs information that is not in the history. The usual object is a hash chain with a signed checkpoint. Fix a hash function h and let

$$h_0 = \text{init}, \quad h_i = h(h_{i-1} \parallel e_i),$$

where e_i is the i -th ledger entry with its event identity. A *checkpoint* is a triple $\text{Anchor}(n, h_n, \sigma)$ in which σ is the sequencer's signature on (n, h_n) .

Proposition 21.6 (Relative completeness). *Suppose an auditor has retained an authentic checkpoint $\text{Anchor}(n, h_n, \sigma)$ independently of the ledger, the signature scheme is unforgeable, and h is collision resistant. If a presented ledger H has hash chain value h_n at position n , then its first n entries match the anchored sequence in length, order, and content, except with negligible probability. A presented ledger shorter than n , or with a different chain value at n , is detected, whether the discrepancy is a truncation, an interior deletion, an insertion, a reordering, or an altered record.*

Proof. The first two cases are direct checks. For the third, two different sequences of n entries hashing to the same h_n yield, at the first position where the chains diverge, a collision in h . The argument is the standard one for hash chains and is used here without further elaboration; its strength is exactly that of the cryptographic assumptions. $\square$

The completeness obtained is relative: relative to the anchor, and to trust in whoever issued it or in the transparency mechanism that holds it. The hypothesis that carries the weight is *independent retention*. A sequencer that can replace both the ledger and the checkpoint can publish a shorter history with a fresh, internally consistent checkpoint, and nothing internal to the pair reveals it. Deployments close that gap by placing checkpoints where the sequencer cannot rewrite them: in an external append-only transparency log of the kind used for web certificates [21], or distributed among independent witnesses. The lesson fits the calculus closely. A history can vouch for the admissibility of what it contains; a claim that nothing is missing needs a witness outside the history whose completeness is in question.

21.4 Duplicate delivery

When records travel between replicas they may arrive more than once. Pair each record with an *event identity* i assigned by the sequencer when the record is appended, so that a ledger is a sequence of pairs (i, ρ) . Extend the replay state with the set I of identities already applied, and let

$$\text{apply}(\langle A, O, I \rangle, (i, \rho)) = \begin{cases} \langle A, O, I \rangle & \text{if } i \in I, \\ \langle \text{apply}(\langle A, O \rangle, \rho), I \cup \{i\} \rangle & \text{otherwise.} \end{cases}$$

Hypothesis 21.7 (Identity coherence). *Any two ledger entries with the same event identity carry the same record.*

Theorem 21.8 (Idempotent disposition). *For every state X and entry e , $\text{apply}(\text{apply}(X, e), e) = \text{apply}(X, e)$. Under Hypothesis 21.7, if $\text{dedup}(H)$ keeps the first occurrence of each identity, then*

$$\text{replay}(H) = \text{replay}(\text{dedup}(H))$$

on the $\langle A, O \rangle$ components, both sides being defined or undefined together.

Proof. The first statement is immediate, since after the first application $i \in I$. For the second, induct on H . An entry whose identity is new is applied on both sides. An entry whose identity was seen earlier is a no-op on the left and absent on the right; by coherence it is the same record as the earlier occurrence, so dropping it cannot change which occurrence was the one applied. $\square$

The mechanism is deliberately simple; any system with an applied-set achieves idempotence. The content of the theorem is in its hypothesis. Exactly-once *disposition* is obtained from at-least-once *delivery* precisely when the sequencer never reuses an identity for a different record, and that is an obligation on the arbiter, not a property the replay procedure can establish for itself.

21.5 Which reorderings replay ignores

Duplicate tolerance concerns the same record arriving twice. A different question is whether distinct records may arrive in a different order.

Definition 21.9 (Independence and dependence order). *Records ρ and σ are independent, written $\rho I \sigma$, when $\text{subj}(\rho) \neq \text{subj}(\sigma)$ and neither is a compensation record whose predecessor is the other's subject. Two histories are trace equivalent, $H \sim_I H'$, when one can be obtained from the other by finitely many exchanges of adjacent independent records. The dependence order $\prec_H$ is the transitive closure of the relation “ ρ precedes σ in H and ρ, σ are not independent”.*

Proposition 21.10 (Replay respects commutation). *If $H \sim_I H'$ then $\text{replay}(H)$ is defined if and only if $\text{replay}(H')$ is, and when defined they are equal.*

Proof. It suffices to treat one exchange, $H = H_1 \cdot \rho \cdot \sigma \cdot H_2$ and $H' = H_1 \cdot \sigma \cdot \rho \cdot H_2$ with $\rho I \sigma$. From the state reached after H_1 , applying ρ reads and writes the map at $\text{subj}(\rho)$, and if ρ is a compensation record, also reads it at the predecessor. By independence, σ writes neither of these keys, and symmetrically. So each application is defined, and produces the same update, whether it comes first or second. The observation components are unions of sets and commute. The remainder H_2 is then applied to equal states. $\square$

Corollary 21.11. *The disposition and observation state recoverable from a ledger is a function of its Mazurkiewicz trace $[H]_{\sim_I}$. Every ancestry chain of Chapters 18 and 19 is a $\prec_H$ -chain.*

Proof. The first sentence is Proposition 21.10. For the second, consecutive records of one projection share a subject and are therefore dependent. $\square$

This result has two consequences for the rest of the book. It makes precise the spine's phrase “ledger order where order represents a true dependency”: the true dependencies are those of $\prec_H$, and the total order imposed by the sequencer carries more information than

replay can use. And it bears on the first comparative claim, since a trace with an independence relation determined by subjects is very close to the configuration structure of an event structure. Chapter 22 takes that comparison up directly, and Chapter 23 asks what the sequencer's extra information is for, given that replay does not need it.

Chapter 22

Petri Nets, Traces, and Event Structures

Chapter plan. Occurrence nets, Mazurkiewicz traces, and prime event structures with causality and conflict. Starting from Proposition 21.10, constructs an event structure from a ledger: events are records, causality is $\prec_H$, and conflict is the relation between two terminal records for one subject, which Corollary 19.2 already shows can never both occur. Tests the first comparative claim and isolates the residue: whether refusal is exactly conflict, and what role the reasons and witnesses play, which an event structure does not carry.

Chapter 23

Logical Clocks, CRDTs, and the Single Sequencer

Chapter plan. Relates happened-before and its linear extensions to the arbiter's sequence. Given that replay depends only on the trace, asks what the total order is needed for: event identity assignment (Hypothesis 21.7), freshness of commitment identities across replicas, and the arbitration of competing compensations. Asks which disposition programs admit a conflict-free replicated implementation.

Chapter 24

Eventual Disposition

Progress (Theorem 15.9) is a statement about one configuration: it is never silently stuck. Eventual disposition is a statement about whole executions: every commitment is eventually refused or collapsed. The second does not follow from the first. An execution can make progress forever while neglecting a particular commitment, a transformation can run forever, a verifier can fail to decide, and an outside party can simply never answer. The theorem of this chapter holds only under hypotheses that exclude each of these, and they are stated as named predicates on executions so that it cannot be read as stronger than it is.

24.1 Hypotheses on executions

An *execution* ξ is a finite or infinite sequence of transitions $C_0 \rightarrow C_1 \rightarrow \dots$; if finite, it is *maximal* when its last configuration has no transition, discharges included. A *redex* is an occurrence of a top-level prefix, tracked from one configuration to the next while the steps taken are elsewhere.

- $\text{Fair}(\xi)$: no redex is enabled at every configuration from some point on without ever being taken. This is weak fairness, a condition on the scheduler; the calculus itself does not schedule.
- $\text{AllTransformsTerminate}(\xi)$: every transformation redex on a bound commitment in ξ evaluates to a value.
- $\text{AllVerifiersDecide}(\xi)$: every verification redex on a transformed commitment in ξ returns `ok` or an authenticated `fail`.
- $\text{AllObligationsResponsive}(\xi)$: every `await` redex in ξ is eventually settled: discharged, or, for the deadline form, discharged or certified as timed out.

The second and third are quantified over the computations the execution actually invokes, not over the lambda layer as a whole. The lambda layer may be Turing complete; the theorem asks only that the particular transformations and verifiers this execution calls on terminate.

24.2 The theorem

Theorem 24.1 (Eventual disposition). *Let $\Sigma; \Gamma; \Delta \vdash C_0 \text{ ok}$ and let ξ be an execution from C_0 that is maximal or infinite and satisfies **Fair**, **AllTransformsTerminate**, **AllVerifiersDecide**, and **AllObligationsResponsive**. Then every commitment that is live at some configuration of ξ is terminal at some later configuration of ξ .*

Proof. Let c be live at C_i and suppose, for contradiction, that it is never terminal afterward. By Theorem 15.5, every C_j with $j \geq i$ is well typed and its linear context contains exactly one right for c .

Ownership. Flatten the process of C_j into its top-level parallel components, looking through restriction. By T-PAR and Lemma 14.1, exactly one component's context contains the right for c ; call it the *owner* O_j . It is not a replication, since T-REP requires an empty context. By T-NIL it is not **0**. So O_j is a prefix or a sum of prefixes.

Persistence. If a prefix of O_j is enabled at C_j , it remains enabled until O_j acts. Its premises concern the states of commitments whose rights O_j holds, which no other component can change, since changing a state requires holding its right; authority facts, which are fixed; terminal facts, which are monotone; freshness, which is always satisfiable; and the results of evaluation and verification, which are deterministic. If the prefix is an **await**, responsiveness supplies its settlement, by discharge or timeout.

Enabledness. By Theorem 15.9 and its proof, every non-await prefix of a well-typed configuration is enabled, given that the computation it invokes terminates, which the second and third hypotheses provide.

Descent. Let $\|P\|$ count the prefix occurrences of P that are not beneath a replication. Each step taken by the owner consumes one prefix, and possibly discards the other summands of a sum, and the right for c passes to a subterm of what remains. Hence the size of the owner of c strictly decreases with each owner step, while steps of other components leave the owner unchanged.

If ξ is infinite, fairness and persistence force the owner to act infinitely often, so its size would decrease without bound, which is impossible. If ξ is finite and maximal, its last configuration has no transition; by responsiveness it is not waiting, since a pending settlement would be a transition; so by Theorem 15.9 it is inert and every commitment is terminal. Either way, c is eventually terminal. $\square$

The proof uses the type system at every step. Ownership is linearity. Persistence depends on ownership: no other component can disable the owner's prefix. Descent depends on T-REP: rights are never replicated, so the owner is a finite term. And the finite case depends on T-NIL: a program cannot end holding a live right.

24.3 What fails without each hypothesis

Each hypothesis is needed, and each has a smallest counterexecution: a well-typed configuration and an execution satisfying every other hypothesis in which some commitment stays live forever. In each case c has just been opened by a principal a with the authority the program uses.

1. *Unfair scheduling.*

$! \text{pop } q' \text{ as } d. \text{refuse}_a d \text{ because } r.0 \mid \text{refuse}_a c \text{ because } r.0.$

A scheduler that always chooses the replicated component takes a step forever and never the refusal of c , although it is enabled throughout. Every commitment the replication creates is disposed of at once; c never moves.

2. *Divergent transformation.* With $A(c) = \text{bound}(q, b)$ and f a fixed point that diverges on b ,

$\text{transform } c \text{ with } f. \text{refuse}_a c \text{ because } r.0.$

The only redex never completes its evaluation premise, and c stays bound.

3. *Nondeciding verifier.* With $A(c) = \text{transformed}(b, v)$ and a verifier g that on (c, v) neither returns a witness nor issues a certificate,

$\text{verify}_a c \text{ using } g \text{ then } \text{refuse}_a c \text{ because } r.0 \text{ else } 0.$

Stability (Hypothesis 1.4) is not enough; the verifier must decide.

4. *Permanently unanswered obligation.*

$\text{await}_e c \omega. \text{refuse}_a c \text{ because } r.0,$

where e never discharges ω . The configuration is well typed and waiting, not stuck in the sense of Theorem 15.9, and the ledger correctly shows nothing. No other component can refuse c , since the awaiting process holds the right.

5. *An environment that neither discharges nor times out.*

$\text{await}_e c \omega. P \text{ within}_b t \text{ else } \text{refuse}_a c \text{ because expired}.0,$

where e never discharges and the clock authority b never certifies expiry. Declaring a deadline does not by itself end the wait; the deadline is only as responsive as the authority that attests to it.

6. *An unattended verification failure.* With $A(c) = \text{transformed}(b, v)$ and a verifier that returns a certified negative result,

$\text{verify}_a c \text{ using } g \text{ then } P \text{ else } \text{await}_h c \omega_{\text{review}}. \text{refuse}_h c \text{ because } r.0,$

where the disposition authority h never completes its review. The commitment sits in `verificationFailed` forever. This is waiting, not a type error, and it exposes the institutional dependency the failed-verification state was introduced to make visible: a negative result that no authorized party acts on.

The last example does not need a hypothesis of its own. In a well-typed program the continuation after a failed verification holds the right, and T-NIL forbids it to stop holding it, so it must either refuse directly, in which case fairness makes the refusal happen, or wait on

an obligation, in which case responsiveness does. What some presentations would state separately as “failed-verification obligations receive an authorized response” is, here, the instance of `AllObligationsResponsive` for the obligation the failed branch awaits. A sixth example shows that typing is needed as well: the untyped process `pop q as c.0` satisfies all four hypotheses and leaves `c` open forever. `T-NIL` rejects it.

The last two examples are where the liveness question meets institutions. A commitment waiting on a party that never answers is not failing in any sense the calculus can detect; the ledger shows it waiting, correctly. A deadline turns silence into a record, `TimedOut`, that says the obligation went unanswered within the declared interval. It does not turn silence into refusal. What follows a timeout, whether further waiting, compensation, or refusal, is decided by an authorized rule in the `else` branch, and choosing that rule is a policy the calculus can express but does not make.

Chapter 25

Bisimulation and the Spectrum as Collapse Rules

Chapter plan. Recasts bisimilarity, and then trace, failures, testing, and ready equivalences, as collapse rules over histories, and compares the result with van Glabbeek’s spectrum. Settles the third comparative claim, using the rule-indexed collapse records of Chapter 12 as the test case.

Chapter 26

Checking the Layer Translation

Chapter plan. With forward simulation and backward adequacy proved at the level of ledgers in Chapter 3, this chapter checks the translation against Gorla’s encodability criteria, where compositionality and name invariance are expected to hold and divergence reflection turns on the reading of transformation. It then attacks the transparent-reading conjecture: that the kernel’s encoding of application agrees with evaluation in the lambda layer. A failure there would not undo the ledger theorems; it would locate precisely the computational content that the kernel does not itself supply.

Chapter 27

Typed Disposition Safety

All the pieces are in place. This chapter states the book's central safety theorem and proves it by a short induction, as the proof spine promised, because each clause has already been established by a local result.

Theorem 27.1 (Typed disposition safety). *Let $\Sigma_0; \Gamma_0; \Delta_0 \vdash C_0 \text{ ok}$, and let $C_0 \rightarrow C_1 \rightarrow \dots \rightarrow C_n$ be any finite execution. Then there are $\Sigma_n, \Gamma_n, \Delta_n$ such that:*

- (1) $\Sigma_n; \Gamma_n; \Delta_n \vdash C_n \text{ ok}$;
- (2) *every live commitment of C_n has exactly one right in Δ_n , and every terminal commitment has none*;
- (3) *every attributed transition of the execution was authorized*;
- (4) *every verified state and every **Verify** record carries a witness valid for its own commitment and value*;
- (5) *every collapsed commitment has a complete ledger path through opening, binding, transformation, verification, and collapse*;
- (6) *every refusal appended during the execution is present in H_n* ;
- (7) *no commitment has two terminal records in H_n* ;
- (8) $\text{replay}(H_n) = \langle A_n, O_n \rangle$;
- (9) *the process of C_n contains no well-typed continuation acting on a commitment whose right its context does not hold, and at most one component holds each right*.

Proof. By induction on n , the case $n = 0$ being the hypothesis. For the step, Theorem 15.5 carries (1) from C_n to C_{n+1} . The remaining clauses are properties of any well-typed, consistent configuration and of the execution leading to it:

- (2) is condition (C3) of (1), that is, the corollary to Theorem 15.5;
- (3) is Theorem 12.9, applied to each step;
- (4) is Corollary 18.3 with Lemma 14.7, and non-transferability under Hypothesis 2.10;

- (5) is Theorem 19.1;
- (6) is Theorem 18.1 with history monotonicity;
- (7) is Corollary 19.2;
- (8) is condition (C2) of (1), maintained by Proposition 21.3;
- (9) is Lemma 15.1, since each prefix acting on c requires c 's right in its context, together with Lemma 14.1, since two components cannot both hold it.

The untyped results used here were proved for executions from a consistent configuration, which (1) guarantees. $\square$

Two things are deliberately absent from the theorem. It says nothing about whether commitments are eventually disposed of; that is Theorem 24.1, and it needs fairness, terminating computations, deciding verifiers, and responsive obligations, none of which a safety theorem should assume. And it says nothing about whether the ledger is complete; that is Proposition 21.6, and it needs a checkpoint retained outside the history.

The three validation levels of the Preface can now be seen together. The laboratory's bounded propositions check the untyped clauses (5), (6), and (7) and the replay clause (8) on every ledger of a finite class, and found the specification errors the book corrected along the way. The untyped proofs establish (3) through (8) for every execution of every program. The typed metatheory adds (1), (2), and (9), which the untyped calculus cannot even state: that rights are neither duplicated nor abandoned, and that no well-typed program contains a competitor for a commitment it does not own.

Chapter 28

A Map of Positions

Chapter plan. One table collecting every comparison: calculus, direction of encoding, criteria met, outcome, and grade of assertion. Answers the three comparative claims in their final form.

Appendix A

Proof Spine

The proof dependency is: unique abstract parsing and alpha-invariance; fresh commitment introduction and lifecycle preservation; append-only history; value and process substitution; subject reduction; authority preservation and binding integrity; witness ancestry; collapse soundness and terminal exclusivity; replay correspondence; compensation conservation; and finally typed disposition safety. Liveness is stated separately because it requires fairness, responsive external dependencies, terminating transformations, and decidable verification. Safety holds for every finite execution without those assumptions.

Result	Status	Location
Unique tokenization	proved	Lem. 6.1
Unique parsing	proved (LL(1))	Thm. 6.2
Printer adequacy	proved	Thm. 6.3
Equivariance	proved	Ch. 5
Commitment identity	proved	Thm. 12.5
Lifecycle and history monotonicity	proved	Ch. 12
Authorized transition	proved (static policy)	Thm. 12.9
Source authenticity of verification failures	proved	Prop. 12.10
Ledger acceptance of verification failures	proved (given certificates)	Prop. 12.11
Diamond for independent steps	proved	Prop. 12.3
Ledger path lemma	proved	Lem. 17.1
Refusal preservation	proved	Thm. 18.1
Binding integrity	proved (exclusivity: typed)	Ch. 18
Witness ancestry	proved	Cor. 18.3
Collapse soundness	proved	Thm. 19.1
Terminal exclusivity	proved	Cor. 19.2
Compensating continuation	proved	Thm. 20.1
Compensation ancestry	proved	Lem. 20.4
Consistency invariant	proved	Prop. 21.3
Replay correspondence	proved	Thm. 21.1
Prefix indistinguishability	proved	Thm. 21.4
Subhistory indistinguishability	proved	Thm. 21.5
Relative completeness	proved (conditional)	Prop. 21.6
Idempotent disposition	proved (under coherence)	Ch. 21
Replay respects commutation	proved	Prop. 21.10
Causal shuffle	proved	Thm. 2.9
Modal terminal exclusivity	proved	Prop. 2.4
No cycles, one re-entry	proved	Prop. 2.12
Kernel translation: forward simulation	proved (ledger level)	Thm. 3.5
Kernel translation: backward adequacy	proved (ledger, universal process)	Thm. 3.7
Kernel translation: transparent reading	conjecture	Ch. 26
Value substitution, linear and unrestricted	proved	Ch. 14
Canonical forms for rights	proved	Lem. 14.7
Preservation	proved	Thm. 15.5
Progress with typed waiting	proved	Thm. 15.9
Protocol fidelity (single commitment)	proved	Thm. 15.10
Typed disposition safety	proved	Thm. 27.1
Eventual disposition	proved (four hypotheses)	Thm. 24.1
Non-derivability of authority	proved (static policy)	Prop. 16.5
Delegation authenticity	proved	Prop. 16.1
Conservation of exclusive capabilities	proved	Prop. 16.2
Preservation under transfer	proved	Prop. 16.3
Delegation chain soundness	proved	Prop. 16.4

Appendix B

Complete Syntax, Typing, and Transition Rules

Chapter plan. The full grammar of both layers, all typing rules, all labelled transition rules, and the replay clause paired with each.

Appendix C

Encodability Criteria, Stated Formally

Chapter plan. Formal statements of the criteria of Chapter 4, with notation fixed for the whole book.

Appendix D

A Worked Encoding: A CCS Fragment into the Calculus

Chapter plan. A full translation of a small CCS fragment with each criterion checked explicitly.

Appendix E

Open Problems

1. *The transparent reading.* The layer translation is proved under the opaque reading, where evaluation and verification results are guard data. Whether the kernel’s encoding of application agrees with evaluation in the lambda layer, so that the kernel itself performs the computation, is open (Chapter 26).
2. *Verifier versioning.* The theorems of Part V assume Hypothesis 1.4. With versioned verifiers, the version must be recorded and witness ancestry restated relative to it; whether an old verification remains evidence under a new version is a policy question the calculus should make explicit rather than decide.
3. *Competing compensations.* Two authorized principals may compensate one terminal commitment. Should the calculus forbid this, record both and require a later arbitration, or leave it to a collapse rule?
4. *Recording the road not taken.* The choice rule leaves no record of the discarded summand. A variant that records it may or may not be more expressive (Chapter 13).
5. *Restriction and visibility.* Restriction scopes naming but not visibility. Whether a scoped ledger is desirable, and what it would do to replay, is open (Chapter 11).
6. *Mutable store.* Replay correspondence excludes S . The exact recording obligations under which S becomes reconstructible remain to be stated.
7. *Denied attempts.* An unauthorized attempt has no transition and leaves no record. Recording `DeniedAttempt` on the annotation channel would make attempted overreach auditable without making it causal; who records it, and whether an untrusted component can flood that channel, is open.
8. *Anchoring the ledger’s end.* Replay cannot detect removal of a ledger’s final records (Chapter 2). The sequencer must publish an anchor; which anchor suffices under which failure model is open.
9. *The purpose of total order.* Replay depends only on the trace (Proposition 21.10). Which guarantees, if any, require the sequencer’s total order beyond identity assignment and freshness (Chapter 23)?

Bibliography

- [1] G. Agha. *Actors: A Model of Concurrent Computation in Distributed Systems*. MIT Press, 1986.
- [2] S. D. Brookes, C. A. R. Hoare, and A. W. Roscoe. A theory of communicating sequential processes. *Journal of the ACM* 31(3), 1984.
- [3] A. Church. *The Calculi of Lambda-Conversion*. Princeton University Press, 1941.
- [4] P. Cousot and R. Cousot. Abstract interpretation: a unified lattice model for static analysis of programs. *POPL*, 1977.
- [5] J.-Y. Girard. Linear logic. *Theoretical Computer Science* 50, 1987.
- [6] D. Gorla. Towards a unified approach to encodability and separation results for process calculi. *Information and Computation* 208(9), 2010.
- [7] C. Hewitt, P. Bishop, and R. Steiger. A universal modular ACTOR formalism for artificial intelligence. *IJCAI*, 1973.
- [8] C. A. R. Hoare. *Communicating Sequential Processes*. Prentice Hall, 1985.
- [9] K. Honda. Types for dyadic interaction. *CONCUR*, 1993.
- [10] L. Lamport. Time, clocks, and the ordering of events in a distributed system. *Communications of the ACM* 21(7), 1978.
- [11] A. Mazurkiewicz. Concurrent program schemes and their interpretations. DAIMI Report PB-78, Aarhus University, 1977.
- [12] R. Milner. *A Calculus of Communicating Systems*. LNCS 92, Springer, 1980.
- [13] R. Milner, J. Parrow, and D. Walker. A calculus of mobile processes, I and II. *Information and Computation* 100(1), 1992.
- [14] M. Nielsen, G. Plotkin, and G. Winskel. Petri nets, event structures and domains, part I. *Theoretical Computer Science* 13, 1981.
- [15] C. Palamidessi. Comparing the expressive power of the synchronous and asynchronous π -calculi. *Mathematical Structures in Computer Science* 13(5), 2003.
- [16] D. Park. Concurrency and automata on infinite sequences. *Theoretical Computer Science*, LNCS 104, 1981.

- [17] C. A. Petri. *Kommunikation mit Automaten*. Dissertation, University of Bonn, 1962.
- [18] G. D. Plotkin. A structural approach to operational semantics. DAIMI FN-19, Aarhus University, 1981.
- [19] F. S. Fatemi Nasrollahi, E. Newby, and R. Albert. Modularity-based approach identifies small sets of control nodes in synthetic and biological Boolean networks. *Scientific Reports* 16:29905, 2026. doi:10.1038/s41598-026-48763-1.
- [20] L. Ding, J. Thomson, J. Taylor, and C. Do. LLMs can assist with proposal selection at large user facilities. *Scientific Reports* 16:29879, 2026. doi:10.1038/s41598-026-60226-1.
- [21] B. Laurie, A. Langley, and E. Kasper. Certificate Transparency. RFC 6962, 2013.
- [22] W. W. Tait. Intensional interpretations of functionals of finite type I. *Journal of Symbolic Logic* 32(2), 1967.
- [23] B. C. Pierce. *Types and Programming Languages*. MIT Press, 2002.
- [24] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski. Conflict-free replicated data types. *SSS*, 2011.
- [25] R. J. van Glabbeek. The linear time–branching time spectrum I. In *Handbook of Process Algebra*, Elsevier, 2001.
- [26] P. Wadler. Propositions as sessions. *ICFP*, 2012.
- [27] G. Winskel. Event structures. In *Advances in Petri Nets*, LNCS 255, Springer, 1987.
- [28] Flyxion. Spherepop specification. Unpublished manuscript, 2025–2026.
- [29] Flyxion. The algebra of binding. Unpublished manuscript, 2026.
- [30] Flyxion. The witness plane: admissibility, verification, and commitment in event systems. Unpublished manuscript, 2026.